

12745146 - Arthur Milani Giovanini
12550991 - João Pedro Arroyo

Sistema de Otimização Semafórica Baseada em Algoritmos Genéticos Multiobjetivo

São Paulo, SP

2025

12745146 - Arthur Milani Giovanini
12550991 - João Pedro Arroyo

Sistema de Otimização Semafórica Baseada em Algoritmos Genéticos Multiobjetivo

Trabalho de conclusão de curso apresentado
ao Departamento de Engenharia de Computa-
ção e Sistemas Digitais da Escola Politécnica
da Universidade de São Paulo para obtenção
do Título de Engenheiro.

Universidade de São Paulo – USP

Escola Politécnica

Departamento de Engenharia de Computação e Sistemas Digitais (PCS)

Orientador: Prof. Dr. Jaime Simão Sichman

Coorientador: Dr. Vinicius Renan de Carvalho

São Paulo, SP

2025

Resumo

O crescimento populacional e a intensificação da urbanização têm elevado significativamente o fluxo veicular nos grandes centros urbanos, resultando em congestionamentos, longos tempos de espera e filas extensas. Nesse cenário, a configuração adequada de semáforos torna-se essencial para garantir maior fluidez e segurança ao tráfego. Soluções inteligentes baseadas em técnicas de Inteligência Artificial apresentam potencial para mitigar esses impactos. Este trabalho propõe o desenvolvimento de um software para otimização de políticas semaforicas em cenários representativos, modelando o problema como uma otimização multiobjetivo resolvida por um algoritmo evolucionário. A abordagem busca minimizar o tempo médio em que os veículos permanecem parados e o maior comprimento de fila observado. Para validar o método e o software desenvolvido, aplica-se a otimização a um cruzamento real no município de Santo André, evidenciando os ganhos obtidos em relação à configuração semaforica atualmente utilizada.

Palavras-chave: otimização multiobjetivo, semáforos, simulação de tráfego, algoritmos evolucionários, sistemas inteligentes de transporte.

Abstract

Population growth and intensified urbanization have substantially increased traffic volumes in major urban centers, leading to congestion, long waiting times, and long queues. In this context, properly configured traffic signals are essential to ensure smoother and safer traffic flow. Intelligent solutions based on Artificial Intelligence techniques offer promising alternatives to mitigate these issues. This study proposes the development of a software tool for optimizing traffic signal policies in representative scenarios, modeling the problem as a multi-objective optimization task solved by an evolutionary algorithm. The approach aims to minimize both the average time vehicles remain stopped and the maximum queue length observed. To validate the proposed method and the developed software, the optimization is applied to a real intersection in the city of Santo André, demonstrating the improvements achieved compared to the current signal configuration.

Keywords: optimization, traffic lights, traffic simulation, algorithms, intelligent transportation systems.

Lista de ilustrações

Figura 1 – Exemplos de movimentos veiculares, mostrando aproximação de auto-móvel e duas possíveis saídas.	16
Figura 2 – Exemplo de movimentos veiculares sobrepostos em uma intersecção . .	17
Figura 3 – Exemplo de semáforo fora de intersecção, devido a tráfego de pedestres	17
Figura 4 – Exemplos de fases de um controlador semaforico.	18
Figura 5 – Movimentos veiculares advindos da direita, relacionados a cada semáforo e indexados.	19
Figura 6 – Movimentos veiculares advindos de baixo, relacionados a cada semáforo e indexados.	19
Figura 7 – Movimentos veiculares advindos da esquerda, relacionados a cada semáforo e indexados.	20
Figura 8 – Exemplo de determinação da região de interesse.	21
Figura 9 – Exemplo de modelagem básica da região definida.	22
Figura 10 – Exemplo de visualização da modelagem completa.	23
Figura 11 – Exemplos comparando filas diferentes com mesma quantidade de veículos retidos.	26
Figura 12 – Função de exemplo e seu respectivo mapa de contorno.	29
Figura 13 – Fronteira de Pareto para um conjunto de pontos de exemplo.	35
Figura 14 – Categorização da população de exemplo em frentes de Pareto	37
Figura 15 – Diagrama representando a relação entre os arquivos usados para configurar uma simulação no SUMO.	44
Figura 16 – Diagrama com arquitetura da solução.	54
Figura 17 – Movimentos possíveis na rede viária	56
Figura 18 – Plano com as fases semaforicas.	58
Figura 19 – Durações das fases semaforicas.	58
Figura 20 – Resumo do Setup Experimental	64
Figura 21 – Fronteiras de Pareto 1800s — Execuções 1 e 2	64
Figura 22 – Fronteiras de Pareto 1800s — Execuções 3 e 4	65
Figura 23 – Fronteiras de Pareto 1800s — Execuções 5 e 6	65
Figura 24 – Fronteiras de Pareto 1800s — Execuções 7 e 8	65
Figura 25 – Fronteiras de Pareto 1800s — Execuções 9 e 10	66
Figura 26 – Fronteiras de Pareto 3600s — Execuções 1 e 2	66
Figura 27 – Fronteiras de Pareto 3600s — Execuções 3 e 4	66
Figura 28 – Fronteiras de Pareto 3600s — Execuções 5 e 6	67
Figura 29 – Fronteiras de Pareto 3600s — Execuções 7 e 8	68
Figura 30 – Fronteiras de Pareto 3600s — Execuções 9 e 10	68

Figura 31 – Fronteira de Pareto final vs Solução Corrente	69
Figura 32 – Fronteira de Pareto final, sem solução corrente	69
Figura 33 – Captura de tela com trânsito em cenário com política semaforica corrente.	73
Figura 34 – Captura de tela com trânsito em cenário com política semaforica do caso 4.2.	73
Figura 35 – Captura de tela do caso 7.1	73

Sumário

1	INTRODUÇÃO	11
1.1	Motivação	12
1.2	Objetivos	12
1.3	Justificativa	12
1.4	Organização do trabalho	13
2	CONTEXTO DO TRABALHO	15
2.1	Otimização de tráfego	15
2.1.1	Conceitos básicos	15
2.1.2	Definição e modelagem da região de interesse	20
2.1.3	Métricas de qualidade do trânsito	24
2.1.4	Problema modelado	27
2.2	Algoritmos genéticos	28
2.2.1	Visão geral	28
2.2.2	Formalização do problema	29
2.2.3	Geração e sobrevivência de populações	30
2.2.4	Seleção: detalhamento do torneio binário	32
2.2.5	Aspectos do cruzamento e da mutação	33
2.3	Algoritmos genéticos multiobjetivo	33
2.3.1	Formalização da otimização: fronteira de Pareto	34
2.3.2	Processo de seleção modificado	34
2.3.3	<i>Ranking</i> de Pareto	35
2.4	Algoritmo NSGA-II	37
2.4.1	Aspectos gerais	37
2.4.2	Crowding distance	38
2.4.3	Seleção por torneio binário	39
2.4.4	Lógica de iteração	41
2.5	Simulador de tráfego SUMO	41
2.5.1	Visão geral	41
2.5.2	Configuração da simulação	42
2.6	Aplicação do algoritmo genético ao problema de otimização	43
2.6.1	Primeira abordagem	44
2.6.2	Segunda abordagem	44
3	MÉTODO DO TRABALHO	47
3.1	Etapas de desenvolvimento	47

3.1.1	Determinação do problema alvo	47
3.1.2	Busca pela solução	47
3.1.3	Levantamento de requisitos	47
3.1.4	Mapeamento de ferramentas disponíveis	47
3.1.5	Planejamento da implementação	48
3.1.6	Implementação do projeto	48
3.1.7	Finalização	48
3.2	Cronograma	48
4	ESPECIFICAÇÃO DE REQUISITOS	51
4.1	Requisitos funcionais	51
4.2	Requisitos não-funcionais	51
5	DESENVOLVIMENTO DO TRABALHO	53
5.1	Tecnologias utilizadas	53
5.2	Arquitetura do sistema	53
5.3	Setup, configuração experimental e plano de execuções	54
5.3.1	Rede viária utilizada: Santo André	55
5.3.2	Determinação de T_S , tamanho de população e quantidade de gerações	60
5.3.3	Definição dos indivíduos e gerações	61
5.3.4	Plano de experimentos	62
5.4	Resultados experimentais	64
5.4.1	Resultado das Execuções de 1800 segundos	64
5.4.2	Resultado das execuções de 3600 segundos	66
5.4.3	Fronteira de Pareto final	69
5.5	Discussão dos resultados	70
5.5.1	Comparação dos tipos de execução	70
5.5.2	Desempenho conjunto das execuções	70
5.5.3	Desempenho individual de execução	71
5.5.4	Comparação visual	72
6	CONSIDERAÇÕES FINAIS	75
6.1	Conclusões	75
6.2	Contribuições	75
6.3	Trabalhos futuros	76
	REFERÊNCIAS	79

APÊNDICES	81
-----------	----

APÊNDICE A – DETALHAMENTO DO <i>NON-DOMINATED SORTING</i> DO NSGA-II	83
--	----

A.1	Análise de complexidade do algoritmo ingênuo	85
-----	--	----

1 Introdução

O crescimento populacional e a intensificação da urbanização têm provocado um aumento expressivo da frota de veículos em circulação nas cidades. Esse fenômeno, aliado à limitação da infraestrutura viária, tem resultado em congestionamentos constantes, maior tempo de deslocamento, consumo excessivo de combustível e aumento significativo na emissão de poluentes. Diante desse cenário, os sistemas de controle de tráfego desempenham papel central na mitigação dos problemas de mobilidade urbana, sendo os semáforos um dos elementos mais importantes para garantir a fluidez e a segurança das interseções.

Tradicionalmente, os tempos de sinalização semafórica são determinados por meio de análises prévias do fluxo de veículos pelos órgãos de trânsito. Esse tipo de configuração baseia-se em ciclos previamente planejados para atender uma demanda média ao longo de períodos do dia. Entretanto, tal método possui limitações: o problema de atribuir tempos semafóricos é extremamente complexo, posto que a alteração de cada semáforo impacta o fluxo recebido por seus vizinhos. Não há, portanto, um algoritmo *greedy* ou alguma fórmula que permita encontrar a política semafórica ótima. Por isso, a presença de políticas semafóricas passíveis de melhoria é esperada. Em um caso extremo, cenários mal configurados podem implicar longos períodos de espera desnecessária para alguns motoristas e, ao mesmo tempo, agravar os congestionamentos. Tais ineficiências apontam para a necessidade de soluções mais inteligentes e automatizadas, capazes de configurar os tempos semafóricos a partir de dados fluxo de demanda.

Nesse contexto, o presente trabalho consiste no desenvolvimento de um software voltado à simulação de tráfego e à otimização do tempo de semáforos em cenários representativos de tráfego urbano. O sistema foi construído com uma arquitetura modular, que permite a modelagem de interseções e do fluxo de veículos e a aplicação de diferentes estratégias de controle de forma desacoplada. Além disso, é possível avaliar o desempenho da política semafórica proposta por meio de métricas quantitativas. A solução desenvolvida modela o problema como uma otimização multiobjetivo e emprega o algoritmo evolucionário NSGA-II (*Non-dominated Sorting Genetic Algorithm*) (DEB et al., 2002) para resolvê-lo. O NSGA-II destaca-se na literatura como um dos algoritmos multiobjetivo mais consolidados, sendo amplamente utilizado na solução de problemas semelhantes ao deste trabalho. Observa-se que, da maneira como o software foi desenvolvido, outros algoritmos poderiam ser usados com pouco esforço de codificação para efetuar a alteração, devido à modularidade.

O software desenvolvido é capaz de gerar dados detalhados a partir de simulações, como tempo médio de espera dos veículos, comprimento das filas e vazão do tráfego em

diferentes intensidades de fluxo. Esses resultados permitem não apenas avaliar a eficácia das estratégias de controle propostas, mas também realizar uma análise crítica sobre sua viabilidade prática, destacando vantagens, limitações e potenciais caminhos para futuras aplicações em ambientes reais. Dessa forma, este trabalho contribui para a discussão sobre soluções inovadoras no gerenciamento do tráfego urbano, oferecendo um protótipo de baixo custo que pode servir como base para estudos mais avançados no campo da mobilidade inteligente.

1.1 Motivação

A motivação para o desenvolvimento deste projeto está diretamente relacionada aos desafios enfrentados pelas cidades modernas no gerenciamento do tráfego. O aumento da frota veicular, aliado à limitação da infraestrutura, exige soluções mais inteligentes que permitam reduzir o tempo de deslocamento e minimizar impactos ambientais. Ao mesmo tempo, já há tecnologias de simulação e otimização propícias ao desenvolvimento de ferramentas de software que podem auxiliar órgãos de trânsito e pesquisadores a avaliar diferentes estratégias de controle antes de sua aplicação prática. O projeto, portanto, é motivado tanto pela relevância social quanto pela oportunidade de aplicar conceitos de engenharia de software, algoritmos de otimização e sistemas inteligentes em um problema de grande impacto.

1.2 Objetivos

O objetivo deste projeto consiste no desenvolvimento de um software capaz de otimizar a política semafórica de uma malha viária qualquer, empregando um algoritmo genético multiobjetivo.

1.3 Justificativa

O tema escolhido é de grande relevância social e acadêmica, pois o gerenciamento eficiente do tráfego urbano impacta diretamente a qualidade de vida da população, reduzindo o tempo perdido em congestionamentos e contribuindo para a diminuição das emissões de gases poluentes. Do ponto de vista acadêmico, o projeto é justificável por integrar conceitos de modelagem de sistemas, algoritmos de otimização, análise de desempenho e desenvolvimento de software, constituindo uma aplicação prática de conhecimentos adquiridos ao longo do curso de Engenharia de Computação. Além disso, a ênfase em uma solução baseada em software garante viabilidade de implementação dentro do escopo do trabalho de conclusão de curso, permitindo resultados concretos sem a necessidade de uma infraestrutura física complexa.

1.4 Organização do trabalho

Esta seção explica a estrutura geral da monografia, abordando os principais pontos de cada capítulo. Após a introdução, tem-se o Capítulo 2, relativo ao contexto do trabalho. Este é dividido em quatro principais blocos: apresentação e modelagem do problema de otimização de tráfego, algoritmos genéticos (e em particular, ao NSGA-II), apresentação do simulador SUMO (*Simulation of Urban Mobility*) ((LOPEZ et al., 2018)) e, por fim, a aplicação do algoritmo genético ao problema de otimização de tráfego construído. Em seguida, no Capítulo 3, correspondente ao método de trabalho, descrevem-se as etapas de desenvolvimento do projeto. O Capítulo 4 descreve os requisitos elencados para o trabalho. Em seguida, o Capítulo 5 descreve o desenvolvimento do trabalho é dividido em duas principais partes. A primeira contém as tecnologias utilizadas, a arquitetura do sistema, e apresenta a motivação e o *setup* dos experimentos efetuados. A segunda apresenta o resultado dos experimentos e sua análise. Por último, no Capítulo 6 são apontadas as conclusões do projeto, suas contribuições e possíveis trabalhos futuros.

2 Contexto do Trabalho

Neste capítulo são construídos os fundamentos requeridos para a apresentação do problema abordado no trabalho e de sua resolução. Primeiro, introduze-se o problema de otimização de tráfego, explicando-se a modelagem tanto da área de interesse, relativo às ruas e aos semáforos, quanto do fluxo de veículos. Em seguida, caracteriza-se como a qualidade do trânsito é metrificada, detalhando-se as métricas usadas na função multiobjetivo. A segunda seção deste capítulo trata de algoritmos evolucionários. Primeiro, são apresentados aspectos gerais e o funcionamento de um algoritmo genético genérico. Em sequência, detalha-se o funcionamento do NSGA-II, com destaque ao *Non-Dominated Sorting* otimizado e ao *crowding distance*. A terceira seção trata do simulador SUMO, e além de mostrar o funcionamento básico do simulador destaca como os aspectos da modelagem do problema apontados na primeira seção do capítulo estão presentes na montagem da simulação. Por fim, a quarta seção do capítulo descreve como o algoritmo genético é usado para resolver o problema abordado neste trabalho.

2.1 Otimização de tráfego

Em termos gerais, o problema abordado por este trabalho é a melhoria do tráfego de veículos em uma região especificada por meio do ajuste do funcionamento dos semáforos dessa região. Assim, o objetivo não é resolver engarrafamentos intrínsecos a uma quantidade excessiva de veículos. Trata-se, portanto, da melhoria do trânsito, dentro do possível, sem alterar o fluxo de veículos na região. Nesta seção, primeiro são apresentados alguns conceitos básicos. Em seguida, define-se de forma precisa como a qualidade do trânsito é metrificada, no escopo deste trabalho.

2.1.1 Conceitos básicos

Em primeiro lugar, é necessário definir os termos relativos à geometria das interseções e à forma como o fluxo de veículos é organizado em seu entorno. Uma **interseção** corresponde ao ponto da rede viária onde duas ou mais vias se cruzam, podendo apresentar diferentes configurações, como cruzamentos simples ou junções com múltiplas aproximações. Cada via que conduz veículos até o cruzamento é denominada uma **entrada**, e é nelas que se formam as **filas** de veículos, isto é, sequências de automóveis aguardando permissão para avançar. Já as vias nas quais os veículos partem do cruzamento são denominadas **saídas**. Dada uma intersecção, um **movimento veicular** é definido como um par dado por uma via de entrada (v_e) e uma via de saída (v_s), e representa que o caminho tomado pelo veículo que realizou este movimento foi atingir o cruzamento por v_e e sair por v_s . A

Figura 1 exemplifica esta definição. O veículo se aproxima do cruzamento em 1a. Em 1b e 1c apresentam-se as possíveis saídas, definindo os dois movimentos veiculares válidos para o veículo.

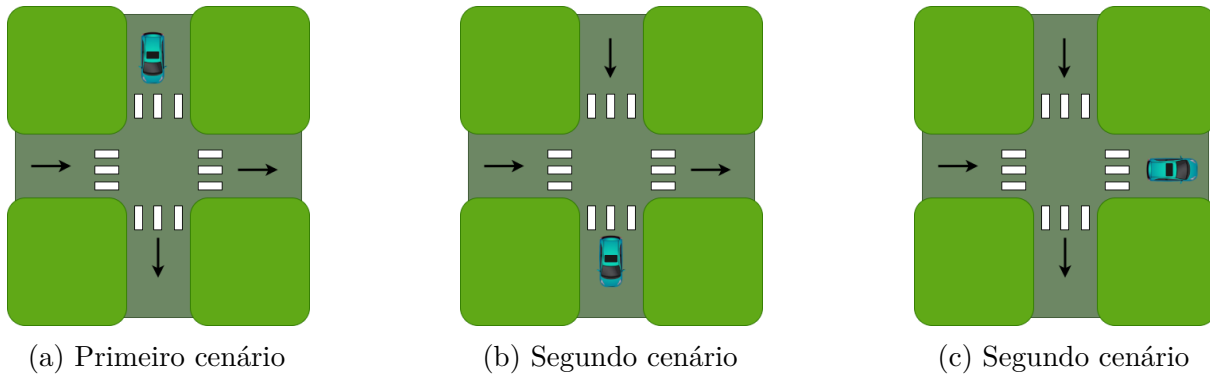


Figura 1 – Exemplos de movimentos veiculares, mostrando aproximação de automóvel e duas possíveis saídas.

Um semáforo é um dispositivo físico, com finalidade de controle de tráfego, que utiliza sinais luminosos para organizar a movimentação de veículos e pedestres, tendo tipicamente os estados verde, amarelo e vermelho. De modo geral, semáforos são instalados em intersecções, posto que devido à sobreposição de movimentos veiculares nem todos podem ser permitidos ao mesmo tempo. A Figura 2 exemplifica este caso. Além disso, semáforos também são instalados para que se garanta um intervalo de tempo no qual pedestres podem atravessar uma determinada via. Por conta deste motivo, também há o caso em que um semáforo não é instalado em uma intersecção, como exemplificado na Figura 3. Por uma questão de simplicidade, para não se precisar dividir algumas explicações em dois casos, deste ponto em diante diz-se que um semáforo é sempre instalado em uma intersecção, e diz-se que a Figura 3 representa uma intersecção do tipo sem intersecção. Todavia, nota-se que há possibilidade de se ter uma intersecção sem semáforos.

Tipicamente, os semáforos são instalados nas entradas das intersecções. Ainda, em termos gerais, os semáforos devem ser programados para que dois movimentos veiculares considerados conflitantes nunca sejam permitidos em um mesmo momento e de modo que haja tempo hábil para os pedestres cruzarem de maneira segura. A primeira destas restrições é crítica: pode-se dificultar significativamente a forma de se montar o problema de otimização semafórica, por implicar restrições adicionais a serem satisfeitas. Se dois semáforos em entradas de uma mesma intersecção são tais que existe saída comum a veículos que entram por tais vias, então seria necessário garantir que nunca ambos estão abertos ao mesmo tempo. Felizmente, há uma forma propícia de enxergar os semáforos para resolver esta questão. Em vez de se pensar em cada semáforo separadamente, é conveniente agregar os semáforos correspondentes a cada intersecção e tratá-los como uma unidade. Neste trabalho, tal é denotado por controlador semafórico, e é associado de forma bijetiva ao conjunto das intersecções. Um controlador semafórico é responsável

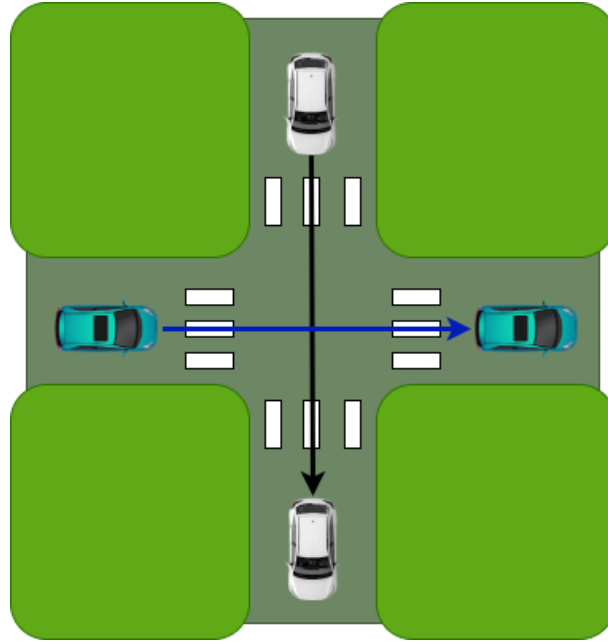


Figura 2 – Exemplo de movimentos veiculares sobrepostos em uma intersecção

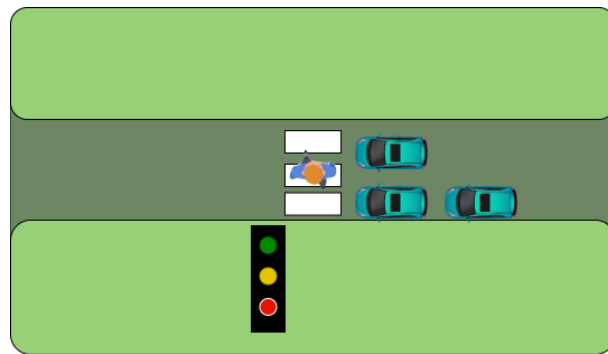


Figura 3 – Exemplo de semáforo fora de intersecção, devido a tráfego de pedestres

por coordenar todos os possíveis movimentos veiculares da intersecção, isto é, define quais trajetos são válidos em termos de pares via de entrada, via de saída.

Para entender como o tratamento por controlador semafórico em vez de semáforos em si simplifica o problema, primeiro é necessário introduzir alguns conceitos. Uma **fase** do controlador semafórico é dada pela classificação de cada movimento veicular possível. Em termos práticos, isso geralmente é representado por uma *string* com caracteres, cada um contido em $\{G, g, y, r\}$. Cada um dos caracteres representa o estado de um movimento veicular. Explica-se o significado de cada caractere.

- **G**: o movimento veicular é permitido e tem prioridade. Aplicável quando: no momento considerado não há outro movimento veicular autorizado que tenha a mesma via de destino; há outro movimento veicular autorizado com mesma via de destino, mas por conta da geometria não há conflito na entrada; há outro movimento veicular autorizado com mesma via de destino e os carros que executam o movimento veicular

marcado com 'G' tem prioridade relativa.

- **g**: o movimento veicular é permitido, porém há outro movimento veicular autorizado em operação que tem a mesma via de destino e os veículos deste outro movimento veicular tem prioridade.
- **r**: o movimento veicular considerado não está autorizado.
- **y**: o movimento veicular considerado está autorizado, mas em uma margem de tempo pequena não estará mais autorizado. Veículos já devem diminuir a velocidade em sua aproximação ao cruzamento.

Ademais, uma forma de se entender o controlador semafórico é imaginar que na verdade há um semáforo para cada movimento veicular. Para o restante desta explicação, toma-se a liberdade de chamar por semáforo o controle de cada um dos movimentos veiculares. Na Figura 4, tem-se quatro exemplos de fases de um controlador semafórico. A Figura 1 apresenta qual é o movimento veicular associado a cada um dos semáforos da Figura 4.



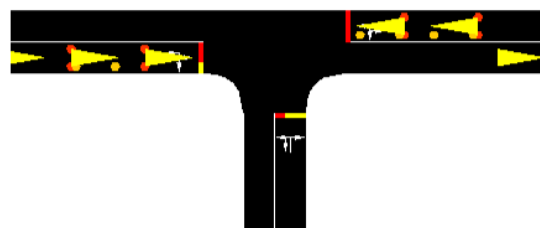
(a) Primeira fase do controlador semafórico.



(b) Segunda fase do controlador semafórico.



(c) Terceira fase do controlador semafórico.



(d) Quarta fase do controlador semafórico.

Figura 4 – Exemplos de fases de um controlador semafórico.

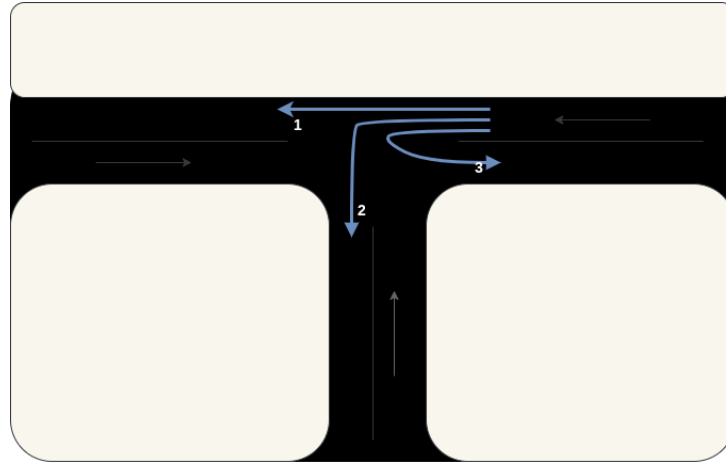


Figura 5 – Movimentos veiculares advindos da direita, relacionados a cada semáforo e indexados.

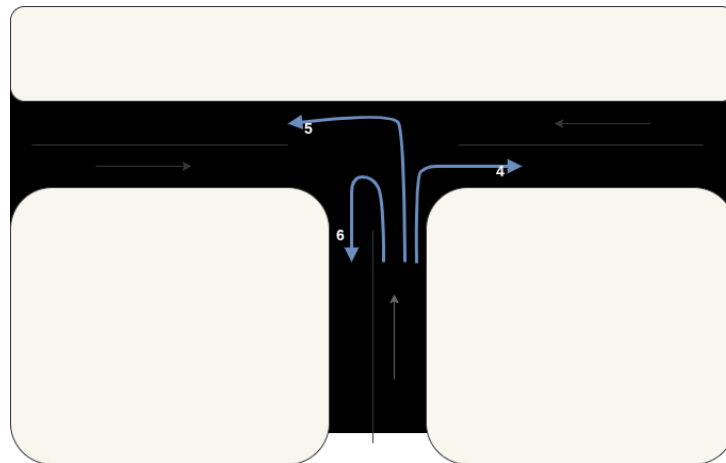


Figura 6 – Movimentos veiculares advindos de baixo, relacionados a cada semáforo e indexados.

Para melhor entendimento, mostra-se uma forma de codificação das fases, condizente com a Figura 4. Usando-se os índices de 1 a 9 definidos nas Figura 5, 6 e 7 como posição na *string* para o estado de cada movimento veicular em cada uma das fases, obtém-se: "GgrrrrGGr", "yyrrrryyr", "rrrGGrGrr", "rrryyryrr". Por exemplo, para o índice 2, que corresponde a vir da direita e sair por baixo, tem-se: na fase vista em 4a está em 'g' (prioridade reduzida) devido ao conflito com o movimento 8; na fase mostrada em 4b, passa a amarelo 'y'; nas fases mostradas em 4c e 4d o movimento não é autorizado - está em 'r'. Já o movimento 8 está autorizado e com preferência nas fases 4a e 4c e está em amarelo nas fases 4b e 4d. Caso fosse o objetivo garantir o cruzamento de pedestres nessa via, uma nova fase com este movimento veicular em 'r' deveria ser adicionada.

Definida a fase de um controlador semafórico, pode-se definir o **plano semafórico** do controlador. Tal é definido como uma sequência finita de pares, cada um formado por uma fase do controlador semafórico e por uma duração, atrelada a um *offset* t_0 . Formalmente, sendo f_i uma fase (por exemplo no formato de *string*) e t_i um valor em segundos, um plano

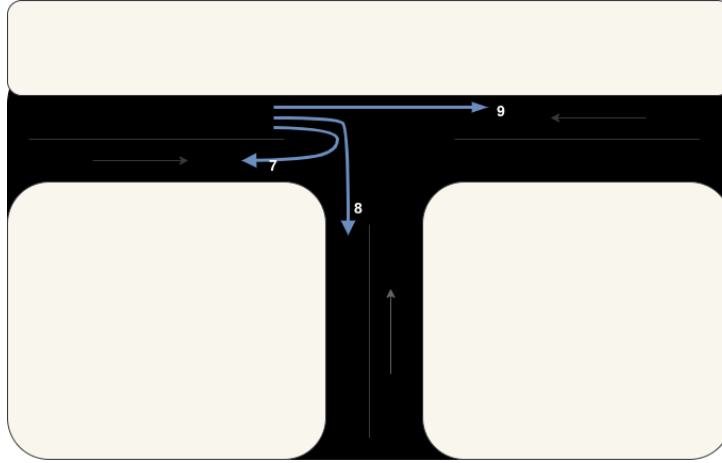


Figura 7 – Movimentos veiculares advindos da esquerda, relacionados a cada semáforo e indexados.

semafórico de n fases P é dado por $P = \{[(f_1, t_1), (f_2, t_2), \dots, (f_n, t_n)], t_0\}$, em que cada fase f_i ocorre por um intervalo de t_i segundos. Disso, pode-se perceber que o funcionamento é cíclico, com período $T_c = \sum_{i=1}^n t_i$. Tal é dito **tempo de ciclo**. O *offset* $t_0 \in [0, T_c]$ indica qual é o ponto inicial de operação do plano semafórico. Isso é bastante importante para manter semáforos sincronizados, possibilitando a progressão contínua do tráfego, o que é comumente denominado por onda verde.

Por fim, faz-se uma classificação das fases f_j de um plano semafórico, necessária para a definição do problema de otimização apresentado em 2.1.4. As fases de um plano semafórico P_i são particionadas em G_i , R_i e Y_i .

- Y_i : uma fase $f_j \in Y_i$ se e somente se algum dos caracteres de f_j é 'y'.
- G_i : uma fase $f_j \in G_i$ se e somente se nenhum dos caracteres de f_j é 'y' e pelo menos um dos caracteres é 'g' ou 'G'.
- R_i : uma fase $f_j \in R_i$ se e somente se todos os caracteres são 'r'.

Apenas as fases pertencentes a G_i tem sua duração como variável de otimização. Todos os demais são fixos, determinados por quem monta o problema.

2.1.2 Definição e modelagem da região de interesse

A subseção anterior explicitou quais aspectos dos semáforos são relevantes ao problema e de que forma são modelados neste trabalho. Isso, todavia, não representa a região de interesse (i.e cujo tráfego se quer melhorar) em sua totalidade. Outros aspectos relevantes são relativos à geometria das vias e aos veículos considerados. Para a montagem do problema de otimização, todos estes pontos devem ser devidamente modelados, ou seja, representados de uma forma simplificada e bem definida.

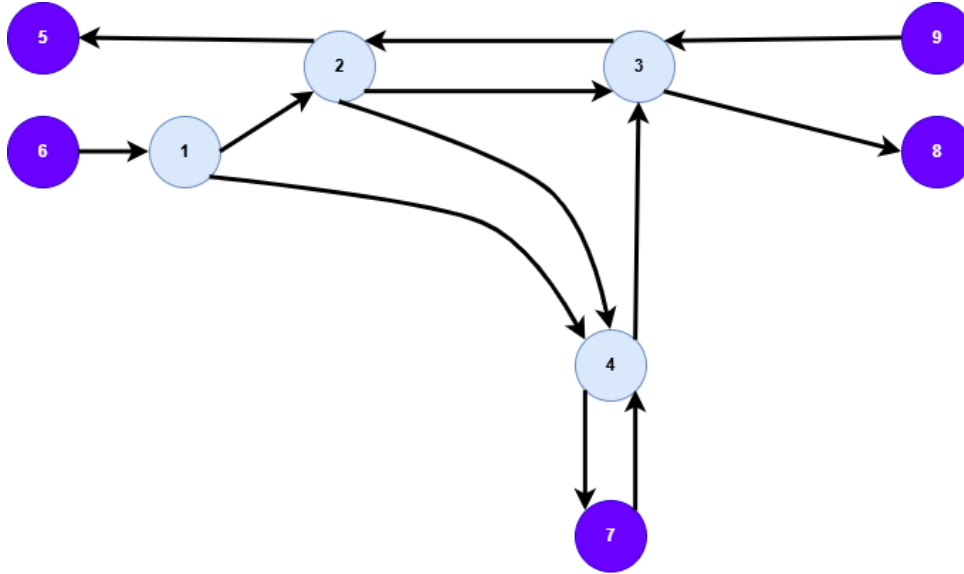


Figura 9 – Exemplo de modelagem básica da região definida.

esta informação envolve definir um conjunto de pontos $\{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$ que represente o formato suficientemente bem por meio de uma aproximação via poligonal. Já para trechos retos (que são o tipo de via do caso de estudo deste projeto), felizmente há uma forma simples de codificar a informação. Em vez de se adicionar informação à aresta, como por exemplo o comprimento da via, é mais fácil adicionar uma informação aos nós: o ponto do plano em que se localizam. Dessa forma, é simples montar o trecho reto que as conecta, e tal já apresenta comprimento similar ao da realidade. Por fim, outros aspectos relevantes tratáveis facilmente apenas pela inclusão de um novo atributo às arestas são: quantidade de faixas (por sentido), largura de faixa, velocidade máxima da via e pontos de parada de ônibus. Por fim, associa-se a cada intersecção um controlador semafórico, com política a ser definida - é justamente o que se altera neste trabalho. Lembra-se que, mesmo sem intersecção física de vias, tratam-se casos como o da Figura 3 como uma intersecção. Além disso, pode-se ter uma intersecção sem semáforos (como exemplo, tem-se a de índice 1 da Figura 8). Na prática, é apenas um controlador semafórico com fase única em que todos os movimentos veiculares autorizados são válidos o tempo todo, potencialmente com discrepância de prioridade. Tais aspectos são considerados suficientes, e partindo-se do modelo dado por 9, já é possível montar uma representação próxima o bastante da realidade. Tal é mostrado na Figura 10, gerada pelo simulador SUMO (*Simulator of Urban Mobility*), detalhado em 2.5.

Finalmente, define-se uma notação para a geometria do modelo. Considera-se que é representado por um grafo $\mathbb{G} = (V, V_{Ex}, E)$ em que V é o conjunto de vértices internos à região, V_{Ex} é o conjunto de extremidades e E é o conjunto de arestas. Há um mapeamento bijetivo entre V e as intersecções da região e entre V_{Ex} e os pontos de extremidade. Também há uma relação biunívoca entre as arestas e o conjunto de vias da região de interesse. Cada nó $v \in V$ ou $v \in V_{Ex}$ é um *label* único e tem como propriedade o ponto

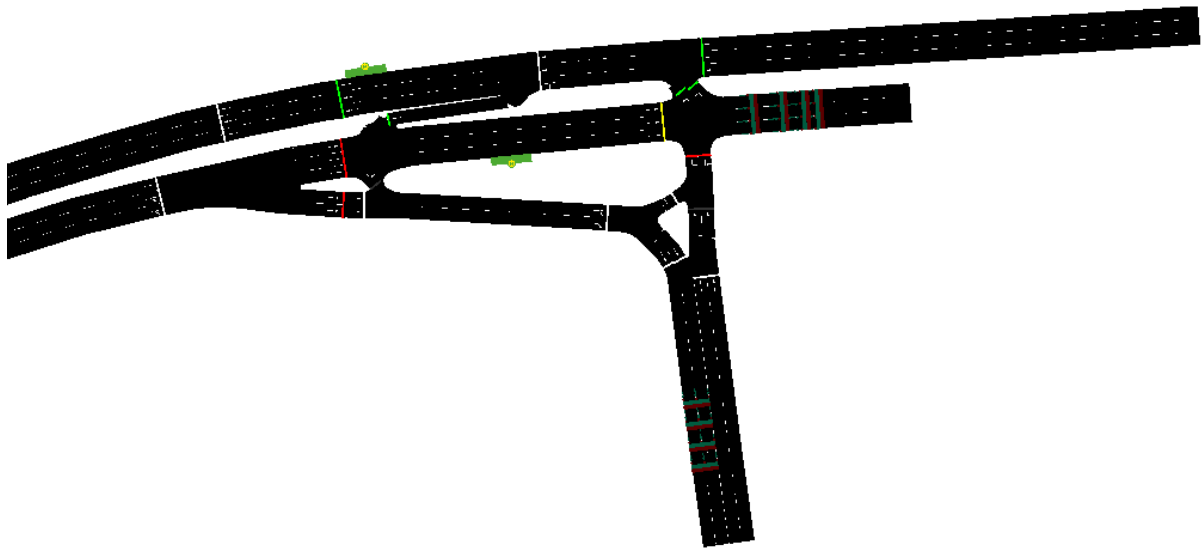


Figura 10 – Exemplo de visualização da modelagem completa.

da região em que se localiza, $v_p \in \mathbb{R}^2$. Para cada aresta $e \in E$, também há um *label* único. A aresta é definida por meio do vértice de origem v_o e de destino v_d : $e = (v_o, v_d)$, com $v_o, v_d \in V \cup V_{Ex}$. Além disso, há atributos adicionais, como a quantidade de faixas, paradas de ônibus, velocidade limite, entre outros, aos quais, por simplicidade, não se atribui notação.

Sobre os veículos da rede

Para modelar o problema, não basta ter um modelo que represente as vias de interesse: é necessário definir o fluxo de veículos na região. Note que a maneira com que os veículos trafegam pela região de interesse é uma função da política semafórica estabelecida, e portanto não se trata de uma informação adicional que se necessita codificar. Assim, exceto pelo estado inicial de veículos na região (i.e veículos por via), o único ponto de interesse é modelar a entrada de veículos na região de interesse. Ainda, a depender do trecho estudado, a seguinte hipótese simplificadora é aplicável: veículos entram e saem apenas por extremidades da região de interesse. As extremidades são definidas como os pontos extremos de vias que não atingem qualquer outra via. Na Figura 8, podem-se observar as seguintes extremidades: 5, 6, 7, 8 e 9.

Com isto, é possível modelar formalmente a entrada de veículos no sistema. De partida, já se devem considerar dois aspectos: a extremidade de entrada e a extremidade de saída do veículo no sistema. Além disso, é necessário determinar, para o sistema como um todo, alguma regra ou algoritmo que define a rota a ser tomada pelo veículo. Todavia, ainda faltam alguns ajustes finos. Primeiro, o tipo de veículo altera o impacto no trânsito. Por exemplo, ônibus precisam realizar paradas intermediárias e ocupam mais espaço, enquanto motos ocupam espaço menor que carros. Portanto, é preciso adicionar um atributo que

especifique o tipo do veículo. Com isso, torna-se possível calibrar o fluxo de veículos para emular uma dada situação não apenas em termos de quantidade de veículos, mas também em composição.

Por fim, deve-se definir o momento em que o veículo entrará no sistema. Como, por razões práticas, não há como simular o trânsito em um período arbitrariamente grande de tempo, este momento não pode ser um número real qualquer. Faz-se necessário introduzir um novo parâmetro, denominado por **tempo de simulação** T_s . Restringe-se o disparo de veículos ao intervalo $[0, T_s]$.

Em termos de notação, diz-se que um plano de rotas P_{route} é um conjunto de automóveis a_i , isto é: $P_{route} = \{a_1, a_2, \dots, a_k\}$. Um veículo a_i pode ser representado por uma tupla: $a_i = (in_i, out_i, type_i, dp_i)$, em que $in_i \in V_{ex}$ é a extremidade de entrada do veículo no sistema, $out_i \in V_{ex} - \{in_i\}$ é a extremidade de saída do veículo no sistema, $type_i$ é o tipo do veículo, podendo ser ônibus, carro ou moto, e $dp_i \in [0, T_s]$ é o momento em que o veículo é inserido no sistema (*departure*). Note que $in_i, out_i \in V_{Ex}$ vem da hipótese simplificadora - poderia ser alterado para $V \cup V_{Ex}$ sem impacto na corretude da formulação do problema.

2.1.3 Métricas de qualidade do trânsito

Até este ponto, apresentou-se de forma geral o problema que se quer resolver e, de forma mais detalhada, definiu-se a região de interesse e o fluxo veicular nela presente. Falta definir, de forma precisa, o que significa um trânsito melhor para este trabalho. Para tal, são levantadas métricas específicas de trânsito, das quais escolhem-se duas para caracterizar o quão bom é o tráfego. Dessa forma, passa a ser possível comparar o desempenho de duas políticas semaforicas diferentes, analisando as métricas elegidas.

Em primeiro lugar, classificam-se as métricas de interesse em dois grupos principais: espaço e tempo. Métricas relacionadas a espaço têm como principal aspecto os comprimentos de filas nas vias, enquanto métricas relacionadas a tempo tem como principal ponto os tempos de viagem dos veículos (tempo dispendido dentro do sistema) ou o tempo em que permanecem parados. Em geral, embora haja correlação entre estas métricas (i.e melhorar uma costuma melhorar a outra e vice-versa) tal não ocorre garantidamente, isto é, não há razão para que sempre uma diminuição do comprimento médio de fila implique um tempo de viagem médio menor, etc. Tal se torna, inclusive, mais aparente conforme aumenta-se a complexidade da rede.

De forma mais detalhada, as principais métricas relativas a tempo seriam: tempo médio de viagem, tempo médio em que veículos passaram parados e máximo tempo de viagem. Matematicamente, denote por lt_i o último momento em que um dado veículo i foi registrado no sistema e por dp_i o instante de tempo em que o i -ésimo veículo entrou

no sistema. Sendo Q a quantidade total de veículos considerados na simulação, o tempo médio de viagem é dado pela equação 2.1.

$$\frac{\sum_{i=1}^Q (lt_i - dp_i)}{Q} \quad (2.1)$$

Analogamente, seja st_i o tempo que o i -ésimo veículo passou parado no sistema, tem-se que o tempo médio em que um veículo permaneceu parado é dado pela equação 2.2.

$$\bar{st}_{sis} = \frac{\sum_{i=1}^Q st_i}{Q} \quad (2.2)$$

Um detalhe importante é que o lt_i pode ser diferente do momento em que o veículo deixou o sistema. Isso ocorre por conta de veículos que, eventualmente, entrem no sistema mas não saiam dele, devido à restrição de tempo de simulação. Nesse caso, sendo T_S o último instante de simulação, tem-se $lt_i = T_S$. É importante contabilizar estes casos, posto que penalizam soluções ruins nas quais veículos ficam presos na rede por um elevado intervalo de tempo. De forma extrema, considere o caso em que um movimento veicular nunca é autorizado. Sem considerar este caso, tais veículos não contribuiriam para aumentar o tempo médio de viagem. Analogamente, o tempo em que cada veículo permanece parado st_i deve ser atualizado ao decorrer do tempo, e não apenas quando os veículos saem do sistema. Por fim, o máximo tempo de viagem é dado por $\max_{i=1}^Q lt_i - dp_i$.

Já para espaço, as principais métricas são o comprimento médio de filas e o comprimento máximo de fila. Denote-se por $q_i(t)$ o tamanho da i -ésima fila no instante de tempo t . Considera-se que há C vias, ao todo, cada uma associada à sua respectiva fila. Ainda, considera-se a avaliação da fila a cada passo discreto, ou seja, os instantes de tempo são $\{t_0 = 0, t_1, \dots, t_k = T_S\}$. Assim, o comprimento médio da i -ésima fila é dado por 2.3.

$$\bar{q}_i = \frac{\sum_{j=0}^k q_i(t_j)}{T_S} \quad (2.3)$$

Finalmente, o comprimento médio de fila do sistema é uma média sobre cada comprimento de fila calculado por 2.3. Tal é mostrado na equação 2.4.

$$\bar{q}_{sis} = \frac{\sum_{i=1}^C \bar{q}_i}{C} \quad (2.4)$$

Já o comprimento máximo de fila é dado pela equação 2.5.

$$q_{max} = \max_{i \in \{1, 2, \dots, C\}, j \in \{0, 1, \dots, k\}} q_i(t_j). \quad (2.5)$$

Outra métrica relevante, que não se enquadra nas duas categorias, é o número de vezes que um veículo precisou parar durante seu trajeto. Tais métricas são comumente empregadas em trabalhos de otimização semaforica, inclusive de temática próxima a este trabalho, envolvendo algoritmos genéticos multiobjetivos. Por exemplo, em (LEAL;

ALMEIDA, 2023) foram utilizados como métricas o número de veículos parados por ciclo e o *delay* médio dos veículos. A mesma decisão quanto às métricas que se busca minimizar foi tomada em (ZHAO HONGXING, 2018). Já em (TEO; KOW; CHIN, 2010), foi usada uma única métrica, correspondente ao comprimento de fila de uma intersecção. Nota-se que em (TEO; KOW; CHIN, 2010) a abordagem lida com apenas uma intersecção, enquanto neste trabalho o modelo proposto é capaz de lidar com uma malha viária qualquer, contendo múltiplas intersecções.

Para este trabalho, escolheram-se as seguintes métricas: minimizar o comprimento máximo de fila q_{max} dado por 2.5, bem como o tempo médio em que os veículos permanecem parados $\bar{st}_{sis}$, dado por 2.2. Além disso, em vez de se adotar uma abordagem que analisa o sistema a cada ciclo semafórico, estas métricas são contabilizadas considerando todo o período de operação $[0, T_s]$. Quanto ao comprimento de fila máximo, é importante ressaltar que é feita uma normalização com base na quantidade de pistas. Considere a Figura 11 para exemplificar o motivo desta decisão.



Figura 11 – Exemplos comparando filas diferentes com mesma quantidade de veículos retidos.

Na Figura 11a, a via tem três faixas e há seis veículos retidos em uma fase cujo sinal está vermelho. Na Figura 11b a quantidade de veículos retidos é a mesma, porém há apenas uma faixa. É evidente que a primeira situação representa uma situação menos problemática, posto que, com a abertura do farol, a vazão de veículos partindo da via será três vezes maior no primeiro cenário, por conta da quantidade de pistas. Por outro lado, se fosse considerada apenas a quantidade de veículos retidos para a fila, ambas as situações seriam metrificadas de forma equivalente. Assim, de maneira mais precisa, a métrica utilizada envolve a profundidade da fila - dois na imagem à esquerda e seis na imagem à direita. Vale notar que, como aproximação, tomando a hipótese simplificadora que os veículos tendem a se distribuir de maneira igual nas pistas (comportamento racional dos motoristas), pode-se aproximar a profundidade da fila $q_i(t)$ pela equação 2.6, na qual $Pistas_i$ é quantidade de pistas da i -ésima via e $SV_i(t)$ é a quantidade de veículos parados na i -ésima via no instante t .

$$q_i(t) = \frac{SV_i(t)}{lanes_i} \quad (2.6)$$

2.1.4 Problema modelado

Com o exposto nas subseções 2.1.2 e 2.1.3, finalmente é possível escrever o problema de otimização formalmente. Considere uma região $\mathbb{G} = (V, V_{Ex}, E)$, com as seguintes enumerações: pontos de intersecção $V = \{v_1, v_2, \dots, v_{|V|}\}$ e vias $E = (e_1, e_2, \dots, e_{|E|})$. Considere um plano de rotas $P_{route} = \{a_1, a_2, \dots, a_{P_{route}}\}$, com veículos a_i disparados no intervalo $[0, T_S]$, sendo T_S o tempo de simulação de interesse e C o total de veículos contidos no plano. Por simplicidade, assume-se que inicialmente o sistema encontra-se vazio, e que T_S é elevado o suficiente para que se atinja um equilíbrio de fluxos. A cada vértice de V associa-se um plano semaforico - os vértices de borda não necessitam de um controlador semaforico, são apenas entradas do sistema. Então, são considerados os planos semaforicos $P_i, i \in \{1, \dots, |V|\}$, associados a cada intersecção v_i . Cada plano semaforico P_i é definido como $P_i = \{[(f_{i,1}, t_{i,1}), (f_{i,2}, t_{i,2}), \dots, (f_{i,L_i}, t_{i,L_i})], t_{i,0}\}$, com tempo de ciclo $T_{c,i}$ fixo. Para cada i , tem-se que L_i é a quantidade de fases determinadas para o correspondente controlador semaforico. Note que o conjunto de fases $f_{i,j}$ (i.e as *strings*) dos controladores é uma entrada do problema de otimização e, portanto, é fixo. São otimizadas apenas as durações $t_{i,j}$ correspondentes às fases de verde, isto é: $t_{i,j} \in G_i$. Considere o vetor com todas as durações de tempos de verde, formado pela duração de cada tempo de verde de cada controlador semaforico, isto é, as variáveis são os elementos de $Gr = \cup_{i=1}^D G_i$. Por simplicidade de indexação, suponha-se que tal vetor é dado por $g_1, g_2, \dots, g_{|Gr|}$. Na prática, cada $g_i = t_{j,k}$ para algum $j \in \{1, \dots, |V|\}$ e $k \in \{1, \dots, L_j\}$. Abstrai-se tal relação entre os índices por meio de mapeamentos m_1 e m_2 , com $g_i = t_{m_1(i), m_2(i)}$. Tem-se que $m_1(i)$ é o índice do controlador semaforico ao qual pertence o tempo de verde i , e $m_2(i)$ é o índice da fase deste controlador semaforico correspondente ao tempo de verde g_i . Assim, o problema pode ser escrito conforme a equação 2.7. Nota-se que, em problemas de otimização, a função que se quer minimizar $\mathbf{F}$ é comumente denominada função objetivo. Outro nome comum é função de aptidão (fitness). Neste caso, por se avaliar mais de uma métrica, diz-se que é uma função multiobjetivo - em oposição a mono-objetivo.

$$\min \mathbf{F}(\mathbf{x}) = \begin{bmatrix} q_{max}(\mathbf{x}) \\ \bar{s}t_{sis}(\mathbf{x}) \end{bmatrix} \quad (2.7)$$

sujeito a:

$$\sum_{j=1}^{L_i} t_{i,j} = T_{c,i}, \quad i \in \{1, \dots, D\} \quad (2.8)$$

$$g_i = t_{m_1(i), m_2(i)} \in (0, T_{c, m_1(i)}) \quad (2.9)$$

$$t_{i,0} \in [0, T_{c,i}) \quad (2.10)$$

onde:

- $\mathbf{x} = [t_{1,0}, t_{2,0}, \dots, t_{D,0}, g_1, g_2, \dots, g_{|G_T|}]$ é o vetor de variáveis de otimização,
- São dados (e fixos) todos os elementos de $\mathbb{G} = (V, V_{Ex}, E)$, exceto pelas variáveis acima enumeradas.

2.2 Algoritmos genéticos

Nesta seção são abordados aspectos gerais de algoritmos genéticos. Primeiro, é dada a intuição desta abordagem e é explicado em quais contextos pode ser útil. Em seguida, são explicados os principais aspectos envolvidos na solução de um problema de otimização via algoritmo genético. Observa-se que ao longo de toda esta seção trata-se apenas do caso mono-objetivo (2.11).

2.2.1 Visão geral

Os Algoritmos Genéticos (GAs) constituem uma classe de métodos de busca e otimização (i.e minimização/maximização de funções) inspirados nos processos naturais da evolução biológica. Introduzidos formalmente por John Holland, na década de 1970, baseiam-se na ideia de que populações de indivíduos submetidas a pressões seletivas e mecanismos de variação tendem, ao longo de gerações, a se adaptar de forma progressiva ao ambiente. Em um contexto computacional, cada indivíduo representa uma solução candidata para um determinado problema, e o ambiente é descrito por uma função objetivo, responsável por medir a qualidade de cada solução.

A motivação por trás do uso de AGs surge principalmente da dificuldade de resolver problemas de otimização complexos. Em tais problemas, pode-se ter uma codificação bem formada, em termos dos parâmetros que definem o problema, eventuais restrições a serem satisfeitas, e a função a ser otimizada. Por outro lado, muitas vezes não há solução determinística computável em tempo computacional viável para o problema apresentado, isto é, não há como determinar a solução ótima por meio de um conjunto de fórmulas ou algoritmos. Nestes casos, é muitas vezes prático utilizar algoritmos genéticos, que justamente dependem apenas da codificação do problema e da capacidade de metrificarmos o quão boa é cada solução.

Na prática, os AGs exploram simultaneamente múltiplas regiões do espaço de busca (domínio do problema) por meio de uma população diversificada, em que se procuram atingir mínimos - nota-se que um dos pontos de atenção é não cair prematuramente em mínimos locais, que podem ser soluções bem piores que o mínimo global do problema. Além disso, por serem estocásticos, podem oferecer soluções robustas ao encontrar, durante a exploração, entradas bastante diferentes que levem a soluções similarmente boas. Esta exploração é baseada em um processo iterativo, no qual uma população de tamanho

especificado, a cada geração (isto é, iteração) gera uma nova população, de tamanho igual, com base em aspectos analisados pelo algoritmo como positivos para a minimização do objetivo (isto é, um cromossomo eficiente) e também em certa aleatoriedade - este processo é denominado mutação. A Figura 12 serve como base para exemplificar a ideia.

Exemplo de superfície com máximos locais e máximo global

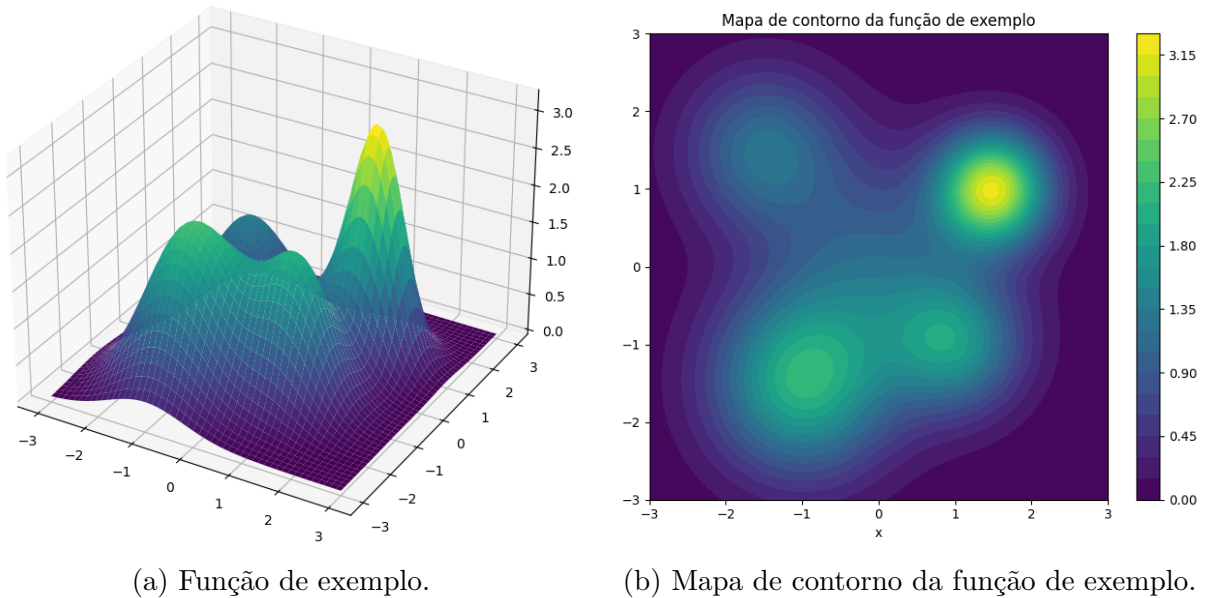


Figura 12 – Função de exemplo e seu respectivo mapa de contorno.

Supondo que se busque maximizar a função de duas variáveis presente 12a, a partir de um determinado conjunto inicial de pontos no plano (i.e população inicial) quer-se explorar o domínio $[-3, 3] \times [-3, 3]$ em busca do máximo local presente em (2, 1). Guiada pela variação da função, a população é levada a máximos locais da função, acompanhando as direções em que há gradiente positivo, até que se anule. A imagem 12b destaca os máximos locais, indicados pelo tom amarelo. Por outro lado, há máximos locais diferentes do máximo global, que podem conduzir a população a uma solução não ótima. Por isso, é necessário garantir uma boa exploração do espaço de soluções, evitando que se fique preso a picos mais baixos. Para funções de mais variáveis e com menor regularidade, o processo de encontrar o máximo global desejado torna-se bastante complexo, salientando-se o *trade off* entre adicionar uma maior tolerância a máximos/mínimos locais, o que permite uma maior exploração do espaço, e uma diminuição na velocidade de convergência - que pode se tornar restritiva, dado que em geral não é computacionalmente viável explorar o espaço solução inteiro.

2.2.2 Formalização do problema

A parte anterior apresentou de forma conceitual o que o algoritmo busca fazer. Nesta parte, de maneira mais detalhada, explicita-se o que os GAs de fato fazem para

buscar por soluções. O primeiro passo para se resolver um problema via GA é codificá-lo no formato requerido por tais algoritmos.

Considere um espaço de busca $\mathbf{D}$. Tal espaço pode ser, por exemplo, um vetor de números pertencentes a um conjunto finito ($\mathbf{D} = \mathbf{A}^k$, $\mathbf{A} = \{a_1, a_2, \dots, a_j\}$), um vetor finito de números reais ($\mathbf{D} = \mathbb{R}^k$), um conjunto finito de estruturas matemáticas (como por exemplo um subconjunto de arestas de um grafo completo de n vértices K_n), etc. Assim, uma solução candidata é qualquer elemento $\mathbf{x} \in \mathbf{D}$.

O AG deve encontrar uma solução que minimize a função objetivo definida pelo problema. Assim, o próximo passo é defini-la. Formalmente, seja $F : \mathbf{D} \rightarrow \mathbb{R}$. Então, o objetivo do problema é dado pela equação 2.11. Note que a caracterização é igual para um problema de otimização de maximização, pois basta definir $g(\mathbf{x}) = -f(\mathbf{x})$ e aplicar a minimização sobre g . Nota-se que o problema definido em 2.11 é denominado mono-objetivo, posto que a função objetivo é composta por uma única métrica. Na seção 2.3 são pontuadas as diferenças em relação ao caso multiobjetivo.

$$\min_{\mathbf{x} \in \mathbf{D}} F(\mathbf{x}) \quad (2.11)$$

2.2.3 Geração e sobrevivência de populações

Com o visto até este ponto, o problema está bem definido. Todavia, ainda não se abordou um ponto vital aos GAs: como são gerados novos candidatos a partir de um conjunto de uma população, de modo a se ampliar o espaço buscado e em direções nas quais espera-se atingir mínimos/máximos melhores.

Para isto, o primeiro ponto é a definição de como o algoritmo genético em si deve armazenar a solução. Já foi definido o espaço de busca $\mathbf{D}$, que formalmente caracteriza a representação da solução. Assim, passa-se ao segundo conceito a ser definido: a população. Em sua execução, o algoritmo genético passa por diversas gerações. Cada geração pode ser caracterizada por um certo instante de tempo t , que por simplicidade será indexado pelos naturais, isto é: $t \in \{0, 1, 2, \dots, t_f\}$ supondo a última geração se dê em $t = t_f$. Assim, a população em um certo instante de tempo t é definida como: $\mathbf{P}_t = (\mathbf{x}_1^{(t)}, \dots, \mathbf{x}_N^{(t)})$. Primeiro, note que o tempo está indexando cada $\mathbf{x}_i$, não se trata de uma função. Segunda, observa-se que foi introduzido o parâmetro N . Tal é o tamanho populacional, e via de regra permanece constante ao longo das gerações. Este parâmetro é muito importante, por conta do seguinte *trade off*: com maior população, em geral consegue-se uma melhor exploração do espaço de busca. Por outro lado, o tempo requerido para a execução do algoritmo genético aumenta. Em particular, para a abordagem feita neste trabalho, a complexidade temporal é linear no tamanho populacional (supondo demais parâmetros constantes).

Com a noção de população bem estabelecida, é possível definir como a geração

seguinte é calculada com base na atual. Primeiro, é necessário definir o conjunto de pais, por meio de uma função de seleção. Intuitivamente, é evidente que não se quer que soluções que apresentaram *fitness* considerado ruim propaguem seus cromossomos, de modo que o uso da função de seleção permite que excluamos o efeito de tais elementos. Além disso, adiciona-se um aspecto probabilístico à função de seleção, o que contribui para ampliar a exploração do espaço solução. Formalmente, define-se: $\mathbb{S} = P_t \rightarrow P'_t \subseteq P_t$. A definição de $\mathbb{S}$ é feita, na prática, com base em uma distribuição de probabilidade que depende de $\mathbf{P}_t$ como um todo e mais especificamente de cada $\mathbf{x}_i$. Há diversas técnicas para implementar esta função, todas com o mesmo princípio: favorecer cromossomos com melhor desempenho.

Em seguida, tem-se o cruzamento/recombinação. São combinados dois ou mais pais pertencentes a $\mathbf{P}'_t$ para criar um novo descendente. Isto é, tem-se uma função $\mathbf{C} : \mathbf{D} \times \mathbf{D} \rightarrow \mathbf{D}$. Assim, por meio de cruzamentos obtém-se $\mathbf{x}' = \mathbf{C}(\mathbf{x}_i, \mathbf{x}_j)$.

O passo seguinte envolve introduzir mutações genéticas, isto é, variabilidade para melhor exploração. Para tal, é necessário um novo operador, com funcionamento de natureza estocástica. Formalmente, a mutação $\mathbf{M} : \mathbf{D} \rightarrow \mathbf{D}$ é uma função que, a partir de uma solução, gera uma nova, inserindo pequenas mudanças, isto é, em uma pequena parte da solução. Por exemplo, se $D = \mathbb{R}^k$ trataria-se de um pequeno conjunto de índices em relação a k .

Com isto, já é possível calcular os candidatos à nova geração. Então, tem-se a substituição da geração atual pela calculada ($\mathbf{P}_{t+1}$). Formalmente, tem-se um operador $\mathbf{R} : (\mathbf{P}''_t, \mathbf{P}'_t) \rightarrow \mathbf{P}_{t+1}$, em que $\mathbf{P}''_t$ são os descendentes gerados pela população $\mathbf{P}_t$, após seleção, cruzamento e mutação. Há possibilidade de se implementar diferentes políticas. Por exemplo, há a troca completa entre gerações (abandona todas as soluções de $\mathbf{P}_t$) ou o elitismo, no qual são selecionados os melhores candidatos, independente de já estarem na população.

Por fim, deve-se definir um critério de parada. Duas possibilidades comuns são critérios relativos à qualidade das soluções (por exemplo, se é notado que o ganho de uma geração para outra não passou de um ϵ pequeno definido por quem montou o problema) ou ao número de gerações (e portanto o número de iterações do algoritmo). Este segundo é bastante útil quando há limitações de tempo para testar, posto que deixar o algoritmo genético executar até que se atinja uma convergência elevada (isto é, os ganhos de uma geração para outra passam a ser insignificantes) pode ser computacionalmente inviável devido ao custo temporal.

Tal processo pode ser codificado com o pseudo-código presente no algoritmo 1.

Algorithm 1 Algoritmo Genético Básico

Require: Espaço de busca $\mathcal{D}$, função objetivo f , população inicial P_0 , tamanho N , número de gerações $T_{\max}$

Ensure: População final $P_{T_{\max}}$ ou conjunto aproximado da fronteira de Pareto

```

1:  $t \leftarrow 0$ 
2:  $P_t \leftarrow P_0$ 
3: while  $t < T_{\max}$  do
4:   for cada indivíduo  $x \in P_t$  do ▷ Avaliação
5:     calcular  $f(x)$ 
6:   end for
7:    $P'_t \leftarrow \text{Selecionar Pais}(P_t)$  ▷ Seleção:  $P'_t \subseteq P_t$ , possivelmente com repetição
8:    $P''_t \leftarrow \emptyset$ 
9:   for cada par  $(x_1, x_2) \in P'_t$  do ▷ Cruzamento
10:     $\text{filhos} \leftarrow \text{Cruzar}(x_1, x_2)$ 
11:     $P''_t \leftarrow P''_t \cup \text{filhos}$ 
12:   end for
13:   for cada indivíduo  $x \in P''_t$  do ▷ Mutação
14:     $x \leftarrow \text{Mutar}(x)$ 
15:   end for
16:    $P_{t+1} \leftarrow \text{Substituir}(P_t, P''_t)$  ▷ Formação da próxima geração
17:    $t \leftarrow t + 1$ 
18: end while

```

2.2.4 Seleção: detalhamento do torneio binário

A seleção é uma das etapas da iteração de um algoritmo genético genérico. Há variadas formas de implementá-la, sendo o método denominado torneio binário um dos mais utilizados - e o padrão para o NSGA-II. Os indivíduos vencedores do torneio binário passam à fase de reprodução, na qual servem como entrada para os operadores de cruzamento e de mutação.

Um torneio binário pode ser definido como uma série de confrontos um contra um, até que um critério de parada seja atingido. Os confrontos são independentes, e o conjunto selecionado para passar à fase de reprodução consiste dos ganhadores de cada confronto. Na verdade, de forma mais precisa, trata-se de um *multiset* - conjunto que permite repetições. Tal ocorre posto que, pela forma como os confrontos são determinados, há possibilidade de uma mesma solução vencer mais de um confronto.

Formalmente, o torneio binário define $\mathbb{S} : P_t \rightarrow P'_t$, em que P'_t é um *multiset* de tamanho igual ao da população, isto é, $|P_t| = |P'_t| = N$. Assim, é necessário executar N confrontos no torneio binário. A seleção dos dois competidores de cada confronto é aleatória. Resta estabelecer o critério que determina o vencedor. Caso trate-se de uma otimização mono-objetivo, a escolha é simplesmente feita com base em qual dos indivíduos minimiza mais a função objetivo. Dessa forma, garante-se o aspecto elitista da seleção. Já para algoritmos multi-objetivo é necessário estabelecer uma métrica adicional.

2.2.5 Aspectos do cruzamento e da mutação

O cruzamento busca explorar novas regiões do espaço solução mantendo características dos pais. Para variáveis contínuas, comumente aplica-se o SBX (*Simulated Binary Crossover*), enquanto para variáveis discretas poderia-se usar o *crossover* de um ou dois pontos. Em geral, são gerados dois filhos por par de pais (i.e por aplicação do operador), mas isso pode ser alterado. Além disso, pode-se adicionar um fator de probabilidade de aplicação, que define a probabilidade de que em vez de fazer o cruzamento, apenas copie-se o par de pais na saída. Já na mutação, busca-se inserir pequenas modificações, evitando-se uma convergência prematura. Para variáveis contínuas, costuma-se usar mutação polinomial ou perturbação gaussiana, enquanto para variáveis discretas é comum usar *bit flips* e trocas. Por fim, para a mutação também é comum se usar uma probabilidade de aplicação, com intuito de não descartar excessivamente boas soluções.

2.3 Algoritmos genéticos multiobjetivo

Uma classe específica de algoritmos genéticos é a MOEA (Multiobjective Evolutionary Algorithms). Diferentemente do apresentados em 2.2, não se quer minimizar uma única função $F : \mathbf{D} \rightarrow \mathbb{R}$, mas sim múltiplas. Assim, é necessário alterar diversos aspectos para adaptar a otimização à minimização de múltiplos objetivos distintos (potencialmente conflitantes) ao mesmo tempo. Nota-se que a nomenclatura muda, e passa-se a denominar o problema por otimização multiobjetivo. Problemas dessa natureza não são incomuns, inclusive no contexto de otimização de tráfego. Por exemplo, no problema abordado neste trabalho quer-se diminuir tanto o comprimento máximo de fila máximo quanto o tempo médio que os veículos passam parados.

A ideia principal de algoritmos MOEA é aproximar a fronteira de Pareto (2.14) e assim evitar o uso de peso ou agregações na função objetivo. Por exemplo, por padrão, caso fosse necessário usar um algoritmo mono-objetivo para lidar com um problema multiobjetivo, seria preciso usar algo como uma média ponderada ao definir f , com pesos determinando o quão relevante é cada componente: $f(\mathbf{x}) = \sum_{i=1}^m c_i \cdot f_i(\mathbf{x})$. Há várias dificuldades nesse formato, a começar por como definir os pesos c_i . Além disso, mesmo com uma boa configuração de pesos, o fato de ser mono-objetivo implica que impreterivelmente será atingida uma única solução, considerada ótima. Na prática, dado os pesos definidos o que se espera é que o algoritmo opte por um dos pontos presentes na fronteira de Pareto do problema. Assim, seria uma abordagem mais limitada. A opção por algoritmos multiobjetivo em relação a mono-objetivos, justamente por possibilitarem aproximar a fronteira de Pareto, é validada pela literatura, como por exemplo em (COELLO; LAMONT; VELDHUIZEN, 2007).

2.3.1 Formalização da otimização: fronteira de Pareto

A maneira de se fazer isto é definir uma função $\mathbf{F}$ multiobjetivo: $\mathbf{F} : \mathbf{D} \rightarrow \mathbb{R}^m$ com $m > 1$. Isto é, tem-se $\mathbf{F}(\mathbf{x}) = \mathbf{F}(f_1(\mathbf{x}), f_2(\mathbf{x}), \dots, f_m(\mathbf{x}))$ com cada f_i definido da forma anterior (mono-objetivo) $f_i : \mathbf{D} \rightarrow \mathbb{R}$. Um ponto de atenção é que no caso mono-objetivo F era imediato comparar a melhor dentre duas soluções: $\mathbf{x}_1$ não é pior que $\mathbf{x}_2$ se e só se: $F(\mathbf{x}_1) \leq F(\mathbf{x}_2)$. Por outro lado, no multiobjetivo é necessário lidar como o caso em que soluções distintas são melhores em componentes diferentes. Este cenário é apresentado pela equação 2.12. Assim, é necessário definir uma nova forma de comparar soluções distintas.

$$\begin{aligned} \mathbf{x}_1, \mathbf{x}_2 \in \mathbf{D}, \quad \exists i \neq j \in \{1, 2, \dots, m\} \\ \text{tais que: } f_i(\mathbf{x}_1) \leq f_i(\mathbf{x}_2), \quad f_j(\mathbf{x}_2) \leq f_j(\mathbf{x}_1) \end{aligned} \quad (2.12)$$

Como exposto acima, o caso multiobjetivo, em geral, não leva a um conjunto unitário, isto é, não há uma solução que é melhor que as demais em todos os m aspectos analisados. Com isto, procura-se na verdade um conjunto minimizador, composto por soluções não dominadas. Intuitivamente, uma solução domina outra se é melhor (isto é, menor) que ela em ao menos uma das m componentes da função $\mathbf{F}$ e não é pior que ela em nenhum componente. Diz-se que $\mathbf{x}_1$ domina $\mathbf{x}_2$ com a seguinte notação: $\mathbf{x}_1 \prec \mathbf{x}_2$. A equação 2.13 define a notação formalmente.

$$\mathbf{x}_1 \prec \mathbf{x}_2 \iff \forall i \in \{1, 2, \dots, m\} \quad f_i(\mathbf{x}_1) \leq f_i(\mathbf{x}_2) \quad \text{e} \quad \exists j \in \{1, 2, \dots, m\} : f_j(\mathbf{x}_1) < f_j(\mathbf{x}_2) \quad (2.13)$$

A Figura 13 exemplifica este conceito. Os pontos em amarelo são os únicos não dominados. Intuitivamente, para o caso $m = 2$, pode-se pensar que um ponto P_1 domina outro ponto $P_2 = (x, y)$ se está contido na região retangular formada pela origem $(0, 0)$, pelo ponto P_2 , e por suas projeções em cada eixo - $(0, y)$, $(x, 0)$. Neste exemplo, o ponto mais à esquerda tem $y < 2.0$ e $x < 10.5$. Com isto, já domina todos com coordenada $y \geq 2$. Dos restantes, apenas o mais à direita é dominado. Tal ponto é dominado pelo que está imediatamente à sua esquerda e ligeiramente abaixo.

Novamente passando a uma notação mais formal, o conjunto de pontos não dominados é caracterizado pela equação 2.14 e denominado fronteira de Pareto.

$$\mathbb{P} = \{\mathbf{x} \in \mathbf{D} : \nexists \mathbf{y} \in \mathbf{D} : \mathbf{y} \prec \mathbf{x}\} \quad (2.14)$$

2.3.2 Processo de seleção modificado

É importante notar que no caso mono-objetivo a seleção por torneio binário usava como critério o desempenho de cada indivíduo na função objetivo. Naturalmente, não é mais possível usar este critério para o caso em que $\mathbf{F}$ é multiobjetivo. Além disso, além de adotar uma nova política que implemente o elitismo, nota-se que surge um outro

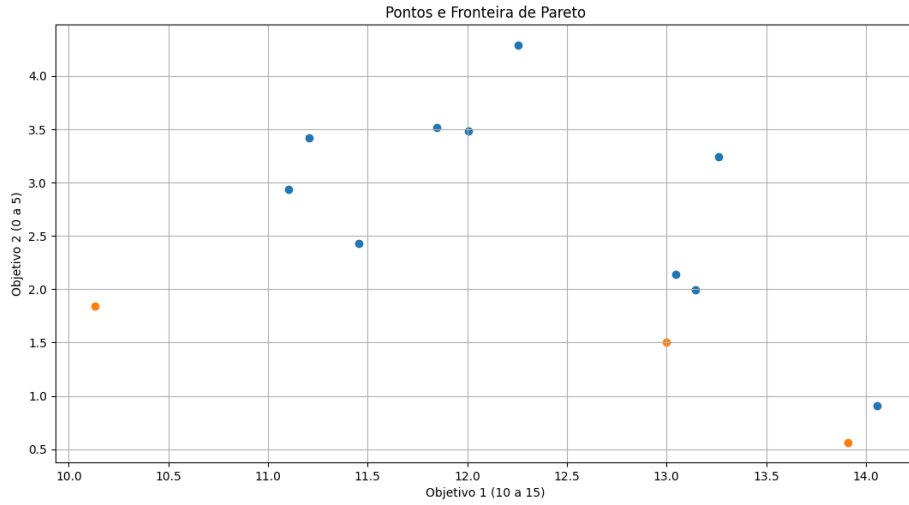


Figura 13 – Fronteira de Pareto para um conjunto de pontos de exemplo.

ponto de interesse: a diversidade de soluções. Como se quer aproximar a fronteira de Pareto $\mathbb{P}$, a população não deve, em geral, convergir a um único ponto. Assim, uma parte importante de algoritmos MOEA é a definição de um critério prático que garanta estes dois aspectos. Em termos de notação, a comparação de diferentes soluções passa a ser feita com um operador $\prec_S$ (diferente de $\prec$), dependente do algoritmo. Apesar de variar conforme o algoritmo, um aspecto comum aos MOEAs modernos é o uso do *ranking* de Pareto como pelo menos uma das partes de $\prec_S$. Tal é explicado adiante.

2.3.3 *Ranking* de Pareto

Como mostrado em 2.3.1, um MOEA busca aproximar a fronteira de Pareto $\mathbb{P}$. Assim, é natural que uma boa métrica seja avaliar o quão próxima uma solução $\mathbf{x} \in \mathbf{D}$ está de $\mathbb{P}$. Tal é justamente o *ranking* de Pareto. O processo de obtenção do *ranking* de Pareto dentro de uma população $\mathbf{P}_t$, comumente denominado *non-dominated sorting*, é descrito adiante.

Para melhor compreensão do funcionamento do algoritmo, apresenta-se um exemplo que serve de acompanhamento às etapas demonstradas. Considere uma população de 16 indivíduos, avaliados em duas métricas f_1 e f_2 , a serem minimizadas. Cada solução é formada por um par, contido no domínio $[0, 5] \times [0, 5]$. A tabela 1 apresenta cada um dos 16 indivíduos presentes em uma geração arbitrária da execução do algoritmo: a primeira coluna associa um *label* a cada ponto, a segunda e terceira mostram o desempenho do indivíduo em cada métrica da função multiobjetivo e a última coluna mostra a solução em si - o ponto no domínio.

A primeira etapa do algoritmo é realizar uma categorização da população em frentes F_1, F_2, F_3 , etc - o *ranking* correspondente ao índice da frente à qual o indivíduo pertence,

Solução	f_1	f_2	Ponto no domínio (x_1, x_2)
A	2	9	(0.5, 4.0)
B	3	7	(1.0, 3.5)
C	4	5	(1.5, 3.0)
D	5	4	(2.0, 2.5)
E	7	3	(3.0, 2.0)
F	8	2	(3.5, 1.5)
I	9	1	(4.0, 1.0)
J	3	6	(1.0, 4.5)
L	6	5	(2.5, 3.0)
M	5	6	(2.0, 4.0)
N	7	5	(3.0, 3.0)
G	6	7	(2.5, 4.5)
K	4	8	(1.5, 5.0)
H	7	8	(3.0, 5.0)
O	8	7	(3.5, 4.5)
P	4	10	(1.5, 0.5)

Tabela 1 – População inicial, valores das funções objetivo e pontos associados no domínio $[0, 5]^2$ para população de exemplo.

sendo melhor um *ranking* mais baixo. Diz-se que um indivíduo pertence à frente F_1 se não existe qualquer outro que o domine. Os pontos pertencentes a F_1 são determinados (de forma ingênua, comparando-se dois a dois) e então realiza-se, com os pontos restantes, o mesmo processo. Dentre os não pertencentes a F_1 , aqueles que não são dominados por nenhum ponto não pertencente a F_1 compõem a frente F_2 , e assim sucessivamente. A equação 2.15 mostra as condições suficientes e necessárias para que um indivíduo $\mathbf{x}_j$ pertença à frente F_i , com $i > 1$.

$$\mathbf{x}_j \in F_i \iff \exists x_k \in F_{i-1} \text{ tal que } x_k \prec x_j \text{ e } \nexists x_m \in F_i \text{ tal que } x_m \prec x_j \quad (2.15)$$

Ainda, tem-se a propriedade: $\forall t > i, \forall x_n \in F_t$ vale $x_j \prec x_n$. A Figura 14 mostra o resultado da categorização para a população de exemplo. Os pontos em vermelho compõem F_1 , e não são dominados por qualquer outro elemento da população. Os pontos em azul compõem F_2 , e são dominados por algum ponto de F_1 : $J \prec J$, $C \prec M$, $D \prec L$. Os pontos em verde compõem F_3 , e cada um é dominado por algum ponto de F_2 : $B \prec K$, $M \prec G$, $L \prec N$. Por fim, os pontos em roxo compõem F_4 , e cada um é dominado por algum ponto de F_3 : $K \prec P$, $K \prec H$, $N \prec O$. Nota-se que as dominações apontadas não são as únicas. Por exemplo, $G \prec O$ já bastaria para O estar em F_4 .

Uma outra forma simples de pensar neste procedimento é: determinar as soluções não dominadas F_1 do conjunto $\mathbf{P}_t$ e repetir o procedimento para $\mathbf{P}_t - F_1$, posto que F_2 é exatamente o conjunto de soluções não dominadas de $\mathbf{P}_t - F_1$, e assim por diante. De modo geral, F_i é o conjunto de soluções não dominadas de $\mathbf{P}_t - \cup_{j=1}^{i-1} F_j$. Nota-se que estas formas de determinar as frentes F_i exigem a comparação dois a dois para a determinação de

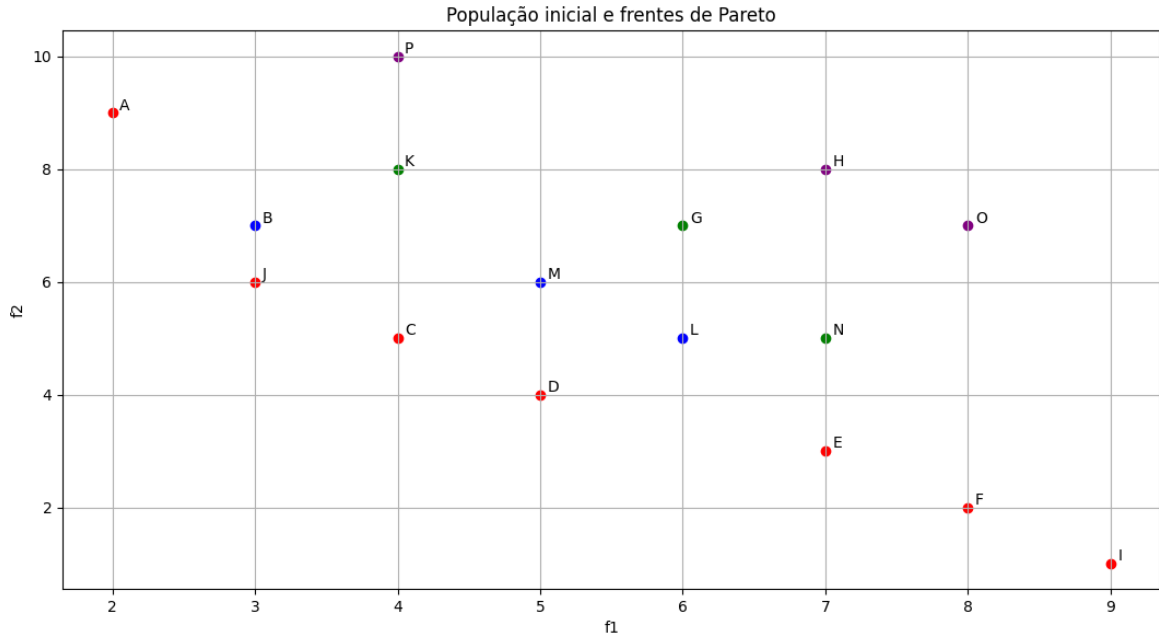


Figura 14 – Categorização da população de exemplo em frentes de Pareto

cada fronteira. Além disso, sendo M a quantidade de objetivos, tem-se que a comparação entre duas soluções é $O(M)$. Em A.1 mostra-se formalmente a complexidade assintótica da abordagem ingênua.

2.4 Algoritmo NSGA-II

2.4.1 Aspectos gerais

O NSGA-II é um dos algoritmos MOEA mais populares, e foi escolhido para resolver o problema de otimização abordado neste projeto. Em primeiro lugar, de maneira geral, são destacados os principais pontos em que este algoritmo se destaca quando comparado aos concorrentes. O primeiro ponto é a velocidade. Publicado em 2002, melhorou a complexidade assintótica temporal do cálculo do *ranking* de Pareto (2.3.3) de $O(M \cdot N^3)$ para $O(M \cdot N^2)$. A forma de se fazer isso é detalhada no apêndice A. Em segundo lugar, eliminou a necessidade de um *sharing parameter*, que em linhas gerais define um raio de similaridade que limita o quão distantes dois elementos da população podem estar para que ainda sejam considerados do mesmo nicho. Com isso, o algoritmo exime o usuário da responsabilidade de passar qualquer parâmetro adicional, sendo suficiente definir o problema de otimização e retornar o cálculo da função multiobjetivo para cada elemento testado. A forma encontrada para tal foi a introdução do *crowding distance*, explicado adiante. Esta métrica permite que a diversidade das soluções seja avaliada, implicando uma melhor exploração do espaço solução. Assim, ao combiná-la com o *ranking* de Pareto,

o algoritmo propõe políticas de caráter elitista, o que permite uma convergência melhor direcionada a soluções ótimas, sem perder uma boa manutenção na diversidade de soluções consideradas.

2.4.2 Crowding distance

Após o agrupamento em diferentes frentes F_i , deve-se calcular a CD (*crowding distance*) de cada elemento. Este fator serve como um desempate na escolha de soluções preferíveis em uma das etapas seguintes, e, em termos gerais, avalia o quão distante é uma solução das demais na mesma frente F_i . Com isso, ao privilegiar-se soluções com maior CD, atua-se na direção de manter uma diversidade de soluções. Inclusive, justamente por isso, a CD de elementos que sejam considerados de borda é inicializada como infinita, como será visto.

Ordenando-se a população como $\{x_1, x_2, \dots, x_N\}$, tem-se que o CD de x_i é dado por $CD_i = \sum_{k=1}^m CD_i^{(k)}$, em que $CD_i^{(k)}$ denota a *crowding distance* para o elemento x_i considerando especificamente o objetivo f_k . Recorda-se que a função multiobjetivo é dada por $(f_1(x_i), f_2(x_i), \dots, f_m(x_i))$ - sendo m a quantidade de objetivos distintos. Assim, falta entender como é calculado o termo CD_i^k . Considerando o k -ésimo objetivo ($f_k(x)$), ordenam-se os x_i com base em $f_k(x_i)$. Denote-se tal vetor ordenado por S_{f_k} . Por exemplo, se $f_k(x_4) < f_k(x_3) < f_k(x_2) < f_k(x_1)$, tem-se a ordem $S_{f_k} = [x_4, x_3, x_2, x_1]$. Denote-se por $\sigma_k(i)$ a posição de x_i em S_{f_k} . Então, a *crowding distance* do indivíduo i relativa à métrica k é dada pela equação 2.16.

$$F(\mathbf{x}) = \begin{cases} CD_i^k = \frac{S_{f_k}[\sigma_k(i)+1] - S_{f_k}[\sigma_k(i)-1]}{\max f_k(x) - \min f_k(x)}, & \text{se } \sigma_k(i) \notin \{1, N\}, \\ CD_i^k = \infty, & \sigma_k(i) = 1 \quad \text{ou} \quad \sigma_k(i) = N \end{cases} \quad (2.16)$$

Note que o segundo caso é definido desta forma justamente porque as bordas devem ser preservadas para se ter diversidade das soluções. Nota-se que a divisão pela diferença entre o máximo e mínimo é uma normalização, feita para não privilegiar indevidamente as métricas f_j que naturalmente apresentam valores maiores em termos absolutos. Por exemplo, se uma métrica f_1 varia em $[1, 100]$ para uma dada população, uma diferença de 0.2 entre $f_1(x_i)$ e $f_1(x_j)$ é muito menos significativa que uma diferença de 0.1 entre $f_2(x_i)$ e $f_2(x_j)$ se esta segunda métrica varia em $[1, 1.5]$ para a população considerada. Por fim, nota-se que caso $\max f_k = \min f_k$, considera-se que esta métrica é não discriminante, de modo que é excluída da análise. Portanto, mais precisamente, $CD_i = \sum_{k \in B} CD_i^k$, em que $B \subseteq \{1, 2, \dots, N\}$ é definido como: $i \in B \iff \max f_i \neq \min f_i$.

Para exemplificar, trabalha-se novamente com a população definida em 1. Para melhor visualização, pode-se consultar a Figura 14, em que são mostradas as frentes de Pareto calculadas para esta população, bem como a ordem em que os indivíduos aparecem

dentro de cada fronteira. Especificamente para F_1 , tem-se as seguintes ordenações: $S_{f_1} = [A, J, C, D, E, F, I]$ e $S_{f_2} = [I, F, E, D, C, J, A]$. Note que para apenas dois objetivos as ordenações necessariamente são o reverso uma da outra. Computa-se que $CD_A = CD_I = \infty$. Já para CD_F considere a equação 2.17 como exemplo didático de cálculo. A tabela 2 apresenta o resultado para todos os 16 pontos considerados no exemplo, usando-se a equação 2.16 para o cálculo.

$$CD_F = \frac{f_1(I) - f_1(E)}{f_1(I) - f_1(A)} + \frac{f_2(E) - f_2(I)}{f_2(A) - f_2(I)} = \frac{9 - 7}{9 - 2} + \frac{3 - 1}{9 - 1} = \frac{2}{7} + \frac{1}{4} \approx 0.5357 \quad (2.17)$$

Ponto	f1	f2	Crowding Distance
A	2	9	∞
B	3	7	∞
C	4	5	0.535714
D	5	4	0.678571
E	7	3	0.678571
F	8	2	0.535714
G	6	7	2.000000
H	7	8	2.000000
I	9	1	∞
J	3	6	0.785714
K	4	8	∞
L	6	5	∞
M	5	6	2.000000
N	7	5	∞
O	8	7	∞
P	4	10	∞

Tabela 2 – Tabela de *crowding distances* para população definida em 1.

2.4.3 Seleção por torneio binário

As duas etapas anteriores, correspondentes à determinação das frentes de Pareto F_i e às *crowding distances* CD_i , são etapas adicionais, responsáveis por pré-processamentos - isto é, não existem por padrão em um algoritmo genético descrito de forma genérica. Finalmente, passa-se a uma etapa padrão para este tipo de algoritmo: a seleção. Tal ocorre pelo método denominado torneio binário, detalhado em 2.2.4.

Recorda-se que o torneio binário define $\mathbb{S} : P_t \rightarrow P'_t$, em que P'_t é um *multiset* de tamanho igual ao da população, isto é, $|P_t| = |P'_t| = N$, por meio de N confrontos. Como abordado em 2.3, a questão é estabelecer o critério que determina o vencedor. Para o NSGA-II, existe um critério padrão. Primeiro, compara-se a frente à qual pertence cada uma das soluções. Por exemplo, sendo x_i e x_j as soluções comparadas, suponha que $x_i \in F_{k_1}$ e que $x_j \in F_{k_2}$. Então, tem-se que x_i vence se $k_1 < k_2$ e que x_j vence se $k_2 < k_1$ - basicamente, privilegia-se a melhor solução. Em caso de empate $k_1 = k_2$, vence aquele

com maior *crowding distance*, ou seja, x_i vence se $CD_i > CD_j$ - e vice-versa. Isso define o operador de comparação $\prec_S$ para o NSGA-II.

Caso o empate persista (por exemplo, confronto 14 de 3), pode-se escolher aleatoriamente o vencedor ou adotar algum critério qualquer, como a escolha pelo menor índice, apenas para não travar a execução. Observe que o desempate pelo *crowding distance*, em oposição ao elitismo claro na escolha por estar contido na frente de Pareto mais próxima da ótima, atua no sentido de aumentar a diversidade de soluções. Por fim, nota-se que há um caso de borda: assumindo seleção aleatória, é possível que o segundo elemento sorteado para o confronto seja igual ao primeiro. Nesse caso, pode-se sortear o segundo elemento até que seja diferente do primeiro. É bastante provável que isso aconteça bem rápido, e o próprio evento de se repetir o indivíduo selecionado é bastante improvável, considerando o elevado tamanho de população que se costuma usar.

Prosseguindo-se com o exemplo 1, foi executada uma simulação do torneio binário implementando a lógica usada no NSGA-II. As respectivas fronteiras F_i e as *crowding distances* da tabela 2 são usadas como entradas. Como já apontado, note que há repetições entre os vencedores. Os resultados obtidos estão presentes na tabela 3. A última coluna determina o *multiset* de indivíduos usados como reprodutores no cruzamento e na mutação. Cada linha representa um confronto, e as células coloridas em amarelo indicam empate da métrica (mesma fronteira ou mesmo CD), enquanto as células coloridas em verde indicam que aquela métrica foi decisiva para a vitória do indivíduo.

Confronto	x_1	Fronteira	CD_1	x_2	Fronteira	CD_2	Vencedor
1	F	1	0.54	K	3	∞	F
2	H	4	2.00	I	1	∞	I
3	B	2	∞	A	1	∞	A
4	N	3	∞	M	2	2.00	M
5	P	4	∞	D	1	0.68	D
6	O	4	∞	G	3	2.00	G
7	C	1	0.54	D	1	0.68	D
8	J	1	0.79	F	1	0.54	J
9	H	4	2.00	I	1	∞	I
10	I	1	∞	J	1	0.79	I
11	K	3	∞	G	3	2.00	K
12	E	1	0.68	F	1	0.54	E
13	M	2	2.00	N	3	∞	M
14	L	2	∞	B	2	∞	B
15	G	3	2.00	J	1	0.79	J
16	M	2	2.00	I	1	∞	I

Tabela 3 – Torneio binário realizado para a população definida em 1.

2.4.4 Lógica de iteração

Caso a iteração do NSGA-II fosse fidedigna a 1, já se teria explicado integralmente o funcionamento do algoritmo. Todavia, como se pode checar em (DEB et al., 2002), este não é o caso. Na verdade, dada uma população P_t o procedimento 1 é utilizado para gerar Q_t , chamado conjunto de filhos de P_t , e não se tem $P_{t+1} = Q_t$. Especificando, Q_t é um *multiset* de tamanho N (igual ao da população original), gerado com os seguintes passos. Primeiro, aplica-se o ranqueamento por fronteira de Pareto e por *crowding distance*. Em seguida, aplica-se um torneio binário aleatório, como descrito na respectiva seção. Por fim, aplicam-se os operadores de cruzamento e mutação, descritos na seção imediatamente anterior. Resta entender de que forma é gerado P_{t+1} a partir de P_t e Q_t .

Primeiro, é gerado o *multiset* Q_t a partir de P_t , como explicado no parágrafo acima. Em seguida, considera-se o *multiset* $R_t = P_t \cup Q_t$. Então, aplica-se o algoritmo de *non-dominated sorting* e calculam-se as *crowding distances*. Note que $|R_t| = 2 \cdot N$. Quer-se selecionar $P_{t+1} \subset R_t$ tal que $|P_{t+1}| = N$. Para tal, adicionam-se as frentes de Pareto a P_{t+1} , a partir da melhor (F_1), até que se atinja o primeiro índice i_l tal que: $\sum_{k=1}^{i_l} |F_k| > N$. Quando isto ocorre, deve-se escolher $N - \sum_{k=1}^{i_l-1} |F_k|$ elementos de F_{i_l} para terminar de preencher P_{t+1} . Isso é feito ordenando-se via *crowding distance*. Selecionam-se os $N - \sum_{k=1}^{i_l-1} |F_k|$ de maior *crowding distance* em F_{i_l} . Com isso, a população do passo seguinte está montada. Note que, das operações necessárias, a de complexidade restritiva é a aplicação do non-dominated sorting, e por isso este processo também é $O(M \cdot N^2)$, mantendo-se a complexidade temporal assintótica do algoritmo. O pseudocódigo 2 apresenta a iteração do algoritmo. Tal foi baseado do artigo original (DEB et al., 2002), e se diferencia da explicação deste texto apenas pelo momento de cálculo de Q_t . Enquanto tal foi feito no começo da iteração na explicação desta seção, no pseudocódigo é feito no final da iteração anterior, isto é, calcula-se Q_{t+1} logo ao final em vez de se calcular Q_t logo no início da iteração anterior.

2.5 Simulador de tráfego SUMO

2.5.1 Visão geral

O SUMO (*Simulation of Urban Mobility*) é um simulador de tráfego *open source*, com controle microscópico e de passos discretos, mantido pela DLR (German Aerospace Center). O controle é microscópico no sentido que cada veículo da rede é modelado de forma independente, isto é, pode-se controlar a rota por ele estabelecida, sua velocidade, aceleração, entre outros. A simulação é uma técnica útil, empregada para se avaliar cenários de tráfego específicos, com risco controlado. Por exemplo, supondo-se que se quer testar uma nova política semafórica, caso o resultado seja ruim, não há prejuízo para o trânsito no mundo real, em oposição a apenas aplicar a nova política sem uma validação prévia.

Algorithm 2 Iteração do NSGA-II

```

1:  $R_t \leftarrow P_t \cup Q_t$  ▷ Combina população de pais e descendentes
2:  $\mathcal{F} \leftarrow \text{fast-non-dominated-sort}(R_t)$  ▷  $\mathcal{F} = (F_1, F_2, \dots)$ 
3:  $P_{t+1} \leftarrow \emptyset$ 
4:  $i \leftarrow 1$ 
   ▷ — Preenchimento da nova população com frentes completas —
5: while  $|P_{t+1}| + |F_i| \leq N$  do
6:    $\text{crowding-distance-assignment}(F_i)$ 
7:    $P_{t+1} \leftarrow P_{t+1} \cup F_i$ 
8:    $i \leftarrow i + 1$ 
9: end while
   ▷ — Seleção parcial da última frente —
10:  $\text{crowding-distance-assignment}(F_i)$ 
11: Ordenar  $F_i$  em ordem decrescente pelo operador  $\prec_n$  ▷ Crowding distance
12:  $P_{t+1} \leftarrow P_{t+1} \cup F_i[1 : (N - |P_{t+1}|)]$ 
   ▷ — Geração da nova população descendente —
13:  $Q_{t+1} \leftarrow \text{make-new-pop}(P_{t+1})$ 
14:  $t \leftarrow t + 1$ 

```

Assim, o objetivo é configurar a simulação para ser o mais próxima possível da realidade, para que os resultados obtidos na simulação (sejam eles positivos ou negativos) sejam de fato traduzidos para o cenário real. Para se avaliar o trânsito na rede estabelecida, há duas principais formas: usar resultados quantitativos, disponibilizados pelo próprio SUMO, como métricas relativas a tempos e filas, ou qualitativamente checar o desempenho, por meio da GUI (*Graphical User Interface*). É fácil notar os trechos que apresentam uma lentidão significativa. Por fim, nota-se que a simulação tem outro ponto positivo além da mitigação de risco, que é a acessibilidade. Qualquer teste em um ambiente não simulado, dentro deste contexto, seria lento e bastante custoso. Por outro lado, o SUMO é um simulador aberto, gratuito, com bastante documentação disponível e atualizações recorrentes. Sem ele ou outros similares, como o VISSIM ([PTV PLANUNG TRANSPORT VERKEHR GMBH, 2025](#)), a realização de projetos como este seria inviável. Em seguida, é apresentado, em linhas gerais, como a simulação é montada e quais são os aspectos sobre os quais o usuário tem controle. Observe que tais são, não à toa, os mesmos ressaltados na seção 2.

2.5.2 Configuração da simulação

A configuração da simulação é o passo correspondente a aplicar o problema modelado sobre o SUMO. Devem ser especificados: a região de interesse, o fluxo de veículos e a política semafórica. No SUMO, a rede é composta por *edges* (trechos de via) conectados por *nodes* (interseções). Cada *edge* possui atributos como comprimento, número de faixas (*lanes*), limite de velocidade, tipo de via e orientação. Já os *nodes* podem conter controladores de tráfego, como semáforos, ou funcionar como cruzamentos não controlados. A configuração

envolve os seguintes passos:

- **Definição da geometria da rede:** utilizando arquivos XML (`.net.xml`), onde são especificados todos os *nodes* e *edges*, incluindo suas coordenadas geográficas e características físicas. Além disso, existe possibilidade de se dividir em dois arquivos: um para nós e outro para intersecção. Como o arquivo `.net` é de difícil confecção e interpretação, em geral, para redes montadas por humanos, é bem mais fácil seguir essa alternativa. Por meio de um comando do SUMO é possível juntar um arquivo `.edge` e um arquivo `.node` para formar um arquivo `.net`.
- **Inserção de controladores de tráfego:** sinais semaforicos podem ser adicionados aos *nodes* por meio de arquivos de configuração de tráfego (`.add.xml`), permitindo a simulação de diferentes estratégias de controle. Além disso, também é possível defini-los no mesmo arquivo que os nós.
- **Especificação de rotas e demanda:** os fluxos de veículos são definidos por rotas (`.rou.xml`), determinando a origem, o destino e o volume de tráfego para cada período de simulação. De forma conveniente, é possível usar uma só cláusula para definir o movimento de diversos veículos, utilizando a opção de tornar o disparo do veículo configurado um evento periódico. Como neste trabalho supõe-se justamente um cenário estático, com fluxos definidos, tais cláusulas simplificam a definição das rotas.
- **Arquivo de configuração:** ajustes como duração da simulação, passo de tempo e nível de verbosidade dos *logs* são configurados para garantir que os resultados reflitam o comportamento do tráfego. Tal é feito por meio de um arquivo `.sumocfg`. Este arquivo também referencia o arquivo de rotas e o arquivo da rede, sendo a base para a montagem da simulação.

Observa-se que a configuração calibrada em relação a um cenário real é de extrema importância, pois permite a geração de métricas de fato representativas do comportamento que o sistema teria se implementado na prática. Os componentes descritos na itemização acima são conectados para que a simulação seja configurada, seguindo a lógica apresentada na Figura 15.

2.6 Aplicação do algoritmo genético ao problema de otimização

Com base nos conceitos e definições apresentados nas seções anteriores, encerra-se este capítulo explicitando como o algoritmo genético multiobjetivo é aplicado ao problema de otimização correspondente, descrito por 2.7. Primeiro é apresentada uma implementação ingênua, traduzida diretamente do problema de otimização. Posteriormente, mostra-se

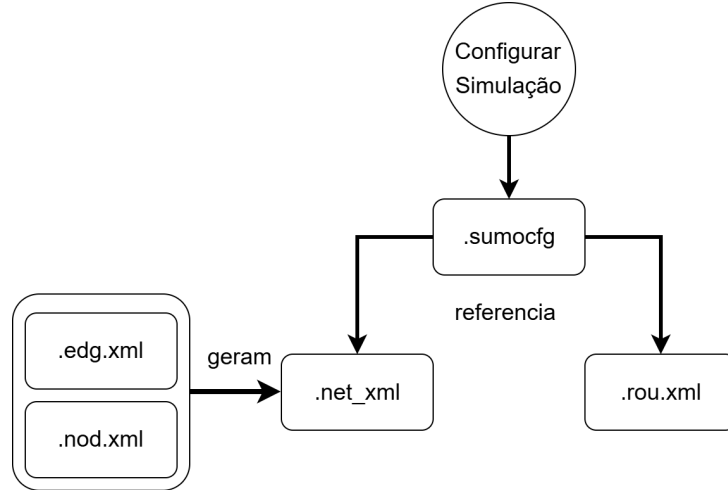


Figura 15 – Diagrama representando a relação entre os arquivos usados para configurar uma simulação no SUMO.

como foi feito na prática, modificando-se a maneira como a restrição 2.8 é aplicada ao problema.

2.6.1 Primeira abordagem

Usando a mesma notação, definem-se os cromossomos do problema (espaço de busca). Na notação de 2.1.4, há $|V|$ cromossomos correspondentes a *offsets* e $|Gr|$ cromossomos correspondentes a tempos de verde. Cada um é um número real não negativo em um intervalo específico, com limitação dada pelas restrições 2.10 e 2.9. Para cada um dos *offsets* $t_{i,0}$ e para cada tempo de verde g_i , a equação 2.18 mostra a notação usada para representar o intervalo ao qual a variável pertence.

$$\begin{aligned} t_{i,0} &\in I_{off_i} = [0, T_{c,i}), \quad i \in \{1, \dots, |V|\} \\ g_i &= t_{m_1(i), m_2(i)} \in (0, T_{c, m_1(i)}], \quad i \in \{1, \dots, |Gr|\} \end{aligned} \quad (2.18)$$

Dessa forma, pode-se expressar o espaço de solução formado pelos cromossomos pelo produto cartesiano mostrado na equação 2.19.

$$\mathbf{D} = \times_{i=1}^{|V|} (I_{off_i}) \times_{i=1}^{|Gr|} (J_{g_i}) \quad (2.19)$$

Como função objetivo, tem-se $F(\mathbf{x}) = (q_{max}(\mathbf{x}), \bar{st}_{sis}(\mathbf{x}))$, definidos pelas equações 2.5 e 2.2, respectivamente. Finalmente, tem-se como restrição restante 2.8. Usando-se o simulador SUMO para calcular a função objetivo para cada indivíduo, tem-se o suficiente para implementar a otimização.

2.6.2 Segunda abordagem

Todavia, a restrição de ciclo de tempo não é particularmente desejável. Como há igualdade, isso significa que é possível reduzir a quantidade de cromossomos do sistema

caso se resolva analiticamente para alguma das variáveis da equação. Mais do que isso, tem-se o seguinte conveniente fato: nenhuma variável de otimização aparece em mais de uma restrição da forma 2.8, posto que cada uma trata do tempo de ciclo de um controlador semafórico distinto. Assim, pode-se lidar com cada restrição de forma independente, e em cada uma isolar algum dos tempos de verde - o que é sempre possível, posto que se assume que não há plano semafórico sem nenhuma fase do classificada como tempo de verde. Outro ponto é que a igualdade é uma restrição muito forte, e ao se trabalhar com *floats* é possível que o processo de mutação não gere uma solução válida, inclusive por erros de representação numérica com precisão limitada. Por isso, optou-se por eliminar um dos tempos de verde de cada controlador das variáveis g_i , isolando-o na sua respectiva restrição. Isso diminui a dimensão do espaço de busca $\mathbf{D}$ significativamente e garante que não haja problema com a geração de soluções inválidas.

Assim, é necessário rever o conjunto de cromossomos $\mathbf{D}$. O objetivo é conseguir remover uma variável referente a cada restrição dada em 2.8 de forma simples e padronizada. De forma prática, toma-se o seguinte critério: para cada controlador semafórico i , o tempo de verde (fase pertencente a G_i) de maior índice é removido. Assim, é necessário determinar tais índices, denotados por lg_i (de *last green*). Para tal, aplica-se a equação 2.20.

$$lg_i = \max_{j \in \{1, \dots, L_i\}} \{j : t_{i,j} \in G_i\}, \quad i \in \{1, \dots, |V|\} \quad (2.20)$$

As variáveis correspondentes a cada índice são isoladas e calculadas em função das demais, conforme a equação 2.21

$$t_{i,lg_i} = T_{c,i} - \sum_{1 \leq j \leq L_i, j \neq lg_i} t_{i,j}, \quad i \in \{1, \dots, |V|\} \quad (2.21)$$

Nota-se que a indexação dada pelos lg_i é relativa à notação $t_{j,k}$, e não aos elementos de $\mathbf{Gr}$, enumerados com uma indexação diferente. Assim, considere o seguinte conjunto auxiliar $\mathbf{LG}$, formado pelos índices variáveis *last green* contidas em $\mathbf{Gr}$. Formalmente, tal é um subconjunto de $\{1, \dots, |Gr|\}$, definido pela equação 2.22. Em linhas gerais, a definição dada por 2.22 é apenas um mapeamento de índices, para que se possa saber quais elementos de $\mathbf{Gr}$ não devem mais ser considerados como variáveis de otimização.

$$\mathbf{LG} = \{i \in \{1, \dots, |V|\} : m_2(i) = lg_{m_1(i)}\}, \text{ dado } g_i = t_{m_1(i), m_2(i)} \quad (2.22)$$

Dessa forma, o espaço de soluções é modificado e passa a ser dado pela equação 2.23.

$$\mathbf{D} = \times_{i=1}^{|V|} (I_{off_i}) \times_{1 \leq i \leq |Gr|, g_i \notin LG} (J_{g_i}) \quad (2.23)$$

Nota-se que o número de variáveis passa de $|V| + |G_r|$ para $|V| + |G_r| - |V| = |G_r|$. Considerando um cruzamento típico (2 entradas e 2 saídas em formato de cruz), cada um teria dois tempos de verde, de forma que: $G_r = 2 \cdot |V|$. Logo, diminuiria-se o número de variáveis de otimização de $3 \cdot |V|$ para $2 \cdot |V|$.

Para que esta abordagem funcione, não é suficiente usar a função objetivo como foi definida em 2.7. Isso decorre do fato que é possível que $t_{i,lg_i} \leq 0$ ao se realizar o cálculo 2.21. Note que o limite superior de $T_{c,i}$ nunca é excedido, dado que as demais variáveis são todas não negativas. Assim, a única preocupação é de fato o limite inferior de cada t_{i,lg_i} . Para resolver isso, adotou-se a modificação dada pela equação 2.24.

$$F(\mathbf{x}) = \begin{cases} (q_{max}(\mathbf{x}), \bar{s}t_{sis}(\mathbf{x})), & \text{se } t_{i,lg_i} > 0 \ \forall i \in \{1, \dots, |V|\}, \\ (\infty, \infty), & \text{caso contrário.} \end{cases} \quad (2.24)$$

Para o caso de baixo de 2.24, sequer é necessário simular via SUMO. Em outras palavras, é introduzido um mecanismo artificial para que o algoritmo não vá em direção a políticas semaforicas inválidas, ou seja, que estariam fora do espaço de busca da abordagem que leva em conta as restrições 2.8. Com isso, a segunda abordagem de aplicação do algoritmo genético ao problema de otimização está integralmente definida. No próximo Capítulo são mostradas as etapas de desenvolvimento adotadas para a implementação do sistema que implementa esta versão do problema de otimização.

3 Método do trabalho

3.1 Etapas de desenvolvimento

3.1.1 Determinação do problema alvo

A primeira etapa do projeto consistiu na busca por um problema real a ser solucionado. Alguns tópicos foram considerados, como sistemas multi-agentes para solução de problemas complexos, sistema de identificação de veículos utilizando Inteligência Artificial e o tema atual: otimização semafórica utilizando IA. Em virtude do impacto de relevância social que o tráfego de veículos têm no país, além de sua viabilidade de implementação (dada a existência de simuladores e ferramentas acessíveis sem custos), optou-se pela otimização semafórica.

3.1.2 Busca pela solução

A segunda etapa do trabalho consistiu em buscar uma solução para o problema de otimização semafórica. Foram abordados cogitadas otimização linear, *deep learning*, algoritmos evolucionários com um único objetivo e algoritmos evolucionários multiobjetivo. Assim, em virtude da natureza do projeto exigir otimização de mais de uma métrica, como tempo de espera e tamanho de fila, optou-se pelo algoritmo multiobjetivo.

3.1.3 Levantamento de requisitos

Como o problema alvo e solução já tinham sido definidos, foi possível iniciar a etapa de definição dos requisitos: a maneira que configuraríamos o projeto para solucionar o problema. Foram definidos os requisitos funcionais e não funcionais do sistema, além da determinação do escopo do projeto: se seria ou não implementado alguma integração com *hardware*, como exemplo a contagem automática de veículos em uma via. Por aumentar expressivamente a complexidade do projeto ao adicionar elementos de *hardware*, optou-se por manter o escopo apenas a nível de *software*.

3.1.4 Mapeamento de ferramentas disponíveis

Com o escopo do projeto definido, a próxima etapa foi a pesquisa pelas ferramentas existentes no mercado, como simuladores de tráfego (SUMO e VISSIM), linguagens de programação (Python, C++), entre outros. Sobre as escolhas principais, o simulador foi o SUMO - um *software* de código aberto, com bibliotecas com integração com Python e simplicidade no uso. A linguagem de programação escolhida foi Python pela vasta disponi-

bilidade de bibliotecas que iriam auxiliar o projeto e, por fim, o algoritmo evolucionário NSGA-II.

3.1.5 Planejamento da implementação

A etapa de planejamento da implementação, foi inicialmente caracterizada pelo aprendizado de uso das ferramentas selecionadas. Como exemplo, como utilizar o NSGA-II com a linguagem Python, funcionamento e configuração do simulador SUMO e bibliotecas de integração de Python com o simulador em questão. Após tal aprendizado, realizou-se um planejamento de como a implementação deveria ocorrer, o que resultou em um diagrama arquitetural de alto nível do projeto. Além disso, foi também planejada as atividades que deveriam ser performadas para a conclusão do projeto, como por exemplo, calibração da rede viária, estudo sobre os parâmetros a serem utilizados, definição de métricas objetivo e comparações a serem realizadas na conclusão. Esta etapa está dividida: parte é abordada nos aspectos teóricos, parte é abordada no desenvolvimento do projeto.

3.1.6 Implementação do projeto

Na etapa de implementação iniciou-se a codificação do projeto seguindo o planejamento da arquitetura, com eventuais mudanças. Mais especificamente, configuração da rede viária que funcionaria como base para as comparações, testes específicos para determinação de certos parâmetros (gerações e tamanho de população, além do tempo de simulação), determinação do *setup experimental* (passo a passo a ser seguido para obtenção dos resultados) e, por fim, execução das simulações, obtenção dos resultados e discussão dos mesmos de acordo com o *setup experimental*.

3.1.7 Finalização

Por fim, com os resultados obtidos, foram realizadas análises críticas do projeto, avaliando tanto sua eficiência geral quanto o desempenho individual das execuções do programa em comparação com o caso tradicional. Além disso, foram apontados possíveis trabalhos futuros que possam contribuir para otimizar ainda mais o software desenvolvido até o momento, bem como recomendações a serem consideradas antes da implementação das soluções em interseções reais.

3.2 Cronograma

O cronograma adotado para o projeto é composto por cinco etapas e encontra-se na tabela 4. Em seguida, as tarefas de cada etapa são detalhadas.

Etapa	1	2	3	4	5
Descrição	Determinação do tema e solução	Levantamento de requisitos	Planejamento	Implementação	Finalização e Entrega
Data	Setembro	Primeira quinzena de Outubro	Outubro e Novembro	Novembro	Dezembro

Tabela 4 – Etapas do projeto.

1. **Etapa 1:** Determinação do tema e solução

- Busca por temas com certa relevância e complexidade
- Determinação de um conjunto de possíveis soluções para solucionar tal problema
- Determinar a solução final a ser abordada para resolver o problema.

2. **Etapa 2:** Levantamento de requisitos

- Mapeamento geral do escopo do projeto
- Determinação dos requisitos funcionais
- Determinação dos requisitos não funcionais

3. **Etapa 3:** Planejamento

- Estudo sobre a utilização do NSGAII com Python
- Familiarização com simulador SUMO e criação de pequenos cenários
- Pesquisa sobre integração do simulador com a linguagem de programação
- Determinar passo a passo a ser realizado para experimentos
- Esquematização da arquitetura do projeto

4. **Etapa 4:** Implementação

- Calibração do cenário tradicional
- Implementação do código Python
- Performar simulações de acordo com o planejamento
- Comparação dos resultados

5. **Etapa 5:** Finalização e Entrega

- Discussão acerca dos resultados do projeto
- Finalização da conclusão
- Revisão completa do relatório
- Polimento final do projeto
- Preparação da apresentação (slides e ensaios).

4 Especificação de Requisitos

4.1 Requisitos funcionais

1. **Otimização semafórica:** O sistema deve otimizar o controle semafórico de uma determinada rede introduzida no sistema alterando os tempos de verde semafóricos e *offsets*.
2. **Execução de simulações:** O sistema deve ser capaz de iniciar uma simulação, dada uma rede viária configura no simulador SUMO, os tempos de verde semafóricos e os *offsets*.
3. **Acesso a resultados anteriores:** O sistema deve permitir o acesso a resultados de execuções performadas anteriormente.
4. **Comparação entre execuções:** O sistema deve permitir a comparação entre resultados de diferentes execuções.

4.2 Requisitos não-funcionais

1. **Usabilidade:** O sistema deve possuir interface simples e intuitiva para configuração dos cenários e execução das simulações.
2. **Desempenho:** O sistema deve ser capaz de simular cenários de tráfego em tempo razoável, mesmo com volumes elevados de veículos. O processamento dos algoritmos de otimização deve ser eficiente o suficiente para viabilizar comparações em múltiplos cenários.
3. **Escalabilidade:** O sistema deve permitir a expansão para cenários mais complexos, incluindo múltiplas interseções conectadas.
4. **Portabilidade:** O sistema deve ser executável em diferentes ambientes de desenvolvimento (Linux e Windows), preferencialmente com dependências de fácil instalação (Python, bibliotecas open-source).
5. **Manutenibilidade:** O sistema deve ser modular, permitindo a inclusão de novos algoritmos ou métricas sem necessidade de grandes modificações no código.
6. **Confiabilidade:** O sistema deve produzir resultados consistentes e reproduzíveis ao executar a mesma configuração de simulação.

7. **Documentação:** O sistema deve ser acompanhado de documentação clara, incluindo instruções de uso, explicação dos módulos e exemplos de execução.

5 Desenvolvimento do Trabalho

5.1 Tecnologias utilizadas

Abaixo, tem-se uma lista com as principais ferramentas utilizadas para a realização do trabalho, listando tecnologias como simulador, linguagem de programação, SGBD e bibliotecas.

- **Linguagem de Programação:** usa-se a linguagem de programação Python (versão 3.11.4) pela facilidade de construção do projeto e pela existência de bibliotecas e frameworks que facilitam a comunicação com o simulador SUMO.
- **SUMO:** foi usado o simulador SUMO (versão 1.24.0) para verificar a eficácia das políticas semaforicas. Tal escolha foi motivada pelo fato do simulador atender as exigências do projeto: possui uma ferramenta de importação de mapas reais e uma biblioteca em Python que possibilita configuração e comunicação dinâmica com a simulação, possibilitando a alteração da política semaforica e a obtenção de métricas.
- **Traci (DLR – German Aerospace Center, 2025):** a biblioteca Traci (versão 1.24.0) foi usada configurar, iniciar e obter os resultados das simulações. A biblioteca permite que o programador tenha controle completo sobre a política de cada controlador, podendo modificar tanto as fases quanto os tempos. Além disso, é possível analisar o sistema a cada passo de tempo da simulação (lembra-se que o SUMO é um simulador de tempo discreto).
- **Pymoo (BLANK; DEB, 2020):** o Pymoo (versão 0.6.1.5) é uma biblioteca Python que implementa diversos métodos de otimização multiobjetivo, incluindo o NSGA-II.
- **Linguagem de Programação (Frontend):** foi usado TypeScript (versão 5.9.3) com o framework Next.js (versão 16.0.0).

5.2 Arquitetura do sistema

Neste tópico é apresentada a arquitetura da solução, descrita pelo diagrama da Figura 16. O usuário deve fornecer o cenário a ser simulado, especificando a geometria da rede a demanda veicular. De maneira mais precisa, tais correspondem aos arquivos mostrados na Figura 15. Por meio destes arquivos, é possível acessar as características da rede a ser otimizada e, com isso, montar o problema de otimização a ser resolvido pelo NSGA-II - descrito em 2.6.2. Por meio da configuração da simulação, descobre-se

quantos são os controladores semafóricos considerados ($|V|$) e quais as fases de cada um. Ao analisar a *string* que representa cada fase, faz-se a classificação nos conjuntos G_i, R_i, Y_i para cada controlador $v_i \in V$. Disso, encontra-se o lg_i (último índice de tempo de verde) e fazem-se os mapeamentos m_1 e m_2 necessários para construir o vetor de variáveis de otimização $\mathbf{x}$. O problema de otimização então finalmente pode ser montado, passando-se as variáveis de otimização limitadas a seu respectivo domínio.

Em seguida, dá-se início ao ciclo de otimização do algoritmo genético. É mantida uma população de soluções de tamanho fixo a cada geração. Em cada iteração do algoritmo, cada uma das soluções da população corrente é avaliada pela função de *fitness*. Isso ocorre por meio da chamada a uma interface que usa a biblioteca Traci, que recebe a solução a ser testada - *offsets* semafóricos e tempos de verde. Por meio do Traci, então, configura-se a simulação com base na política semafórica a ser testada e dispara-se a simulação, executada pelo simulador SUMO. Com a conclusão da simulação, as métricas de interesse são obtidas pela interface. Em seguida, o tempo médio de espera e o tamanho máximo de fila são retornados ao NSGA-II. Após obter a avaliação de cada elemento da população, o NSGA-II executa o algoritmo descrito em 2 para gerar uma nova população. O ciclo se repete, até que seja processado um número especificado de gerações.

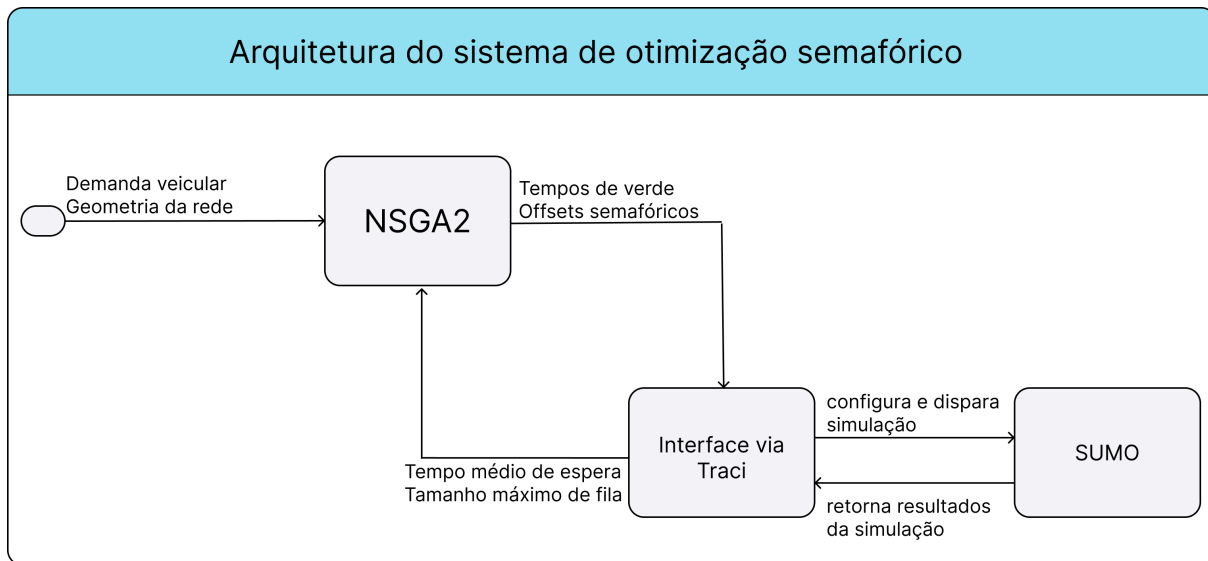


Figura 16 – Diagrama com arquitetura da solução.

5.3 Setup, configuração experimental e plano de execuções

Para validar o conceito do projeto, bem como sua correta implementação, realizaram-se experimentos para uma rede viária correspondente a um trecho de Santo André, com dados reais de fluxo de veículos, advindos de (PROJETO..., 2023). Além disso, (PROJETO..., 2023) especifica o plano de fases dos semáforos, bem como a política semafórica corrente. Com isso, é possível montar o problema de otimização correspondente

ao trecho e comparar o resultado com o obtido pela política corrente. Assim, falta apenas definir os parâmetros a serem usados na simulação: T_S , o tamanho da população e a quantidade de gerações.

Tais parâmetros foram definidos com base em um estudo prévio acerca do tempo de execução, detalhado em 5.3.2. Embora originalmente T_S fosse configurado para 1 hora (3600 segundos), posto que os fluxos medidos em (PROJETO..., 2023) foram referentes a este intervalo de tempo, notou-se que o elevado tempo de execução do programa poderia dificultar a realização do experimento - que, para ser representativo, precisa ser repetido várias vezes. O gargalo do sistema foi mapeado e notou-se que se trata do tempo para executar a simulação no SUMO. Sendo N o tamanho da população e X a quantidade de gerações, tem-se $N \cdot X$ execuções de simulação do SUMO. Assim, levando-se em conta o gargalo do sistema, a complexidade é multilinear sobre estes parâmetros. Por isso, tais também precisaram ser limitados. No estudo prévio, fizeram-se algumas execuções para estimar o tempo médio requerido por cada simulação do SUMO. Além disso, usando-se T_S progressivamente menores fez-se um estudo para avaliar se as soluções encontradas com $T_S < 3600$ mantinham desempenho parecido quando simuladas com $T_S = 3600$ ou se as métricas eram significativamente impactadas. Este estudo é explicitado mais adiante no documento e seu resultado mostrou que não seria possível usar uma simulação com $T_S < 1800$ segundos. Ainda, estimando-se o tempo de execução de cada simulação, concluiu-se que 30 indivíduos e 50 gerações eram parâmetros adequados, tanto por não implicarem um tempo de simulação restritivo quanto por serem suficientes para se obter soluções que atingem a fronteira de Pareto.

5.3.1 Rede viária utilizada: Santo André

Para garantir que o algoritmo genético desenvolvido seja útil em cenários reais, não só em fictícios, é ideal a utilização de simulações compatíveis com a realidade. Em virtude disso, foi necessário calibrar uma rede viária no simulador SUMO de acordo com a realidade, utilizando dados fornecidos pelo departamento de transporte da Escola Politécnica da USP. O cenário utilizado localiza-se no município de Santo André - São Paulo, em um cruzamento entre as Avenidas São Paulo e Avenida Giovanni Battista Pirelli e os dados obtidos correspondem ao horário das 17:15 às 18:15.

O processo de verificação de calibragem da rede consiste em analisar os dados de fluxo coletados em pontos diferentes e compará-los com os resultados da simulação. A seguir, a Figura 17 mostra as trajetórias possíveis dos veículos no trecho considerado. Além disso, os postos de coleta de informações são indicados pelos símbolos em verde e em vermelho com formato de localização. A Tabela 5 mostra os fluxos veiculares medidos.

A princípio, para se calibrar um cenário, é preciso determinar o fluxo de veículos entrando na rede e a configuração de diferentes rotas tomadas pelos veículos. Na Figura

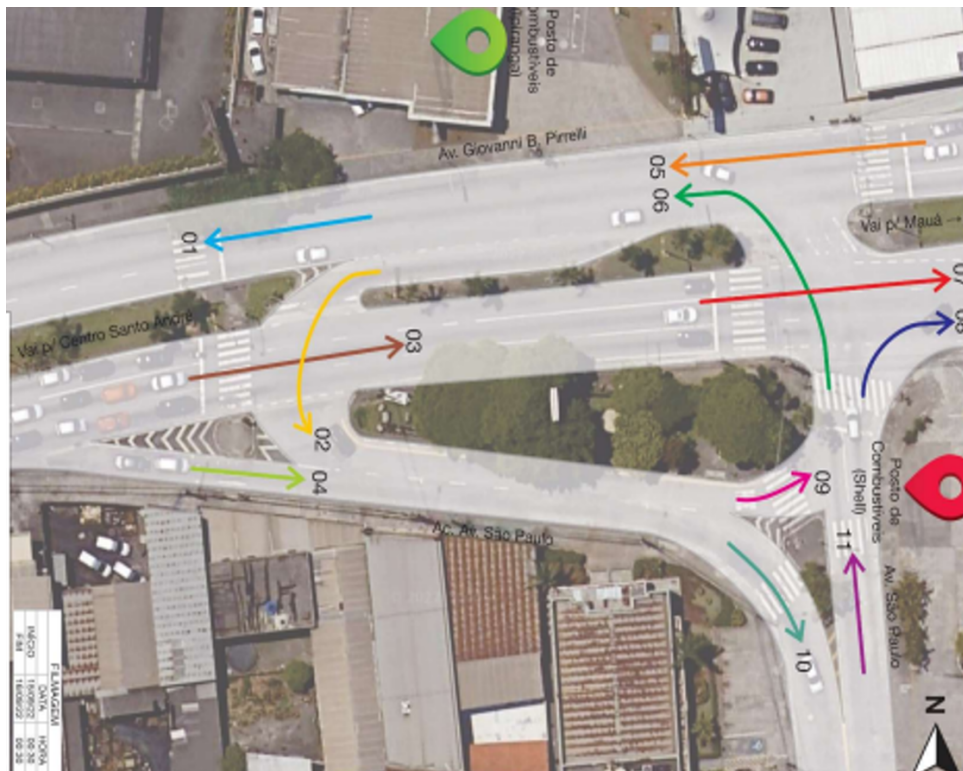


Figura 17 – Movimentos possíveis na rede viária

	MOTO	CARRO	ÔNIB	CAM
M1 Tarde	370	2260	24	69
M2 Tarde	39	208	0	5
M3 Tarde	716	2271	23	49
M4 Tarde	78	325	9	6
M5 Tarde	350	2119	18	69
M6 Tarde	65	349	8	3
M7 Tarde	716	2271	22	50
M8 Tarde	18	101	1	3
M9 Tarde	13	53	0	0
M10 Tarde	106	477	9	12
M11 Tarde	64	424	9	5

Tabela 5 – Dados de fluxos veiculares no horário de pico 17:15 - 18:15

17, nota-se que os veículos precisam tomar certas decisões em alguns pontos: por exemplo, os veículos que estão no movimento M04 optam entre os movimentos M09 e M10. As proporções de veículos que optam pelos caminhos M09 e M10 são definidas com base nos dados da tabela acima e tais proporções possibilitam a determinação da quantidade de veículos tomando cada uma das rotas. Os resultados das proporções em cada ponto de decisão são mostrados nas Tabelas 6, 7 e 8.

Definidas as proporções, é possível estimar a quantidade de veículos que executam cada movimento, tornando possível determinar o número aproximado de veículos em cada uma das rotas na rede viária. As rotas e seus respectivos fluxos configurados no simulador

Veículo	M10 (%)	M09 (%)
Moto	90	10
Carro	90	10
Ônibus	100	0
Caminhão	100	0

Tabela 6 – Proporções veiculares entre movimentos 09 e 10

Veículo	M06 (%)	M08 (%)
Moto	78	22
Carro	78	22
Ônibus	100	0
Caminhão	50	50

Tabela 7 – Proporções veiculares entre movimentos 06 e 08

Veículo	M01 (%)	M02 (%)
Moto	90	10
Carro	92	8
Ônibus	100	0
Caminhão	93	7

Tabela 8 – Proporções veiculares entre movimentos 01 e 02

são mostrados na Tabela 9.

Origem	Destino	Moto	Carro	Ônibus	Caminhão
Av Giovanni (Leste)	Av São Paulo	70	292	9	6
Av Giovanni (Leste)	Av Giovanni (Leste)	716	2264	23	50
Av Giovanni (Leste)	Av Giovanni (Oeste)	8	32	0	0
Av Giovanni (Oeste)	Av São Paulo	31	153	0	5
Av Giovanni (Oeste)	Av Giovanni (Leste)	4	17	0	0
Av Giovanni (Oeste)	Av Giovanni (Oeste)	315	1946	24	64
Av São Paulo	Av São Paulo	5	26	0	0
Av São Paulo	Av Giovanni (Leste)	14	93	0	2
Av São Paulo	Av Giovanni (Oeste)	45	305	0	3

Tabela 9 – Fluxos entre Origem e Destino (veh/h)

Em relação aos ônibus, pela quantidade ser relativamente baixa em relação aos demais fluxos, foi realizada uma simplificação: os mesmos possuem apenas 3 rotas: Avenida Giovanni (Sentido Leste) - Avenida Giovanni (Sentido Leste), Avenida Giovanni (Sentido Leste) - Avenida São Paulo e Avenida Giovanni (Sentido Oeste) - Avenida Giovanni (Sentido Oeste).

Outro ajuste essencial para calibragem da rede é a configuração dos tempos

semafóricos de acordo com a realidade. Para tal, foram obtidos de (PROJETO..., 2023) os dados semafóricos do cruzamento entre as Avenidas Giovanni e São Paulo. A Figura 18 apresenta as fases dos semáforos, enquanto a Figura 19 apresenta a duração programada para cada fase. Nota-se que as indicações R , r , G e Y têm significados diferentes da definição apresentada aos caracteres $\{G, g, r, y\}$ em 2.1.1. Nesta figura, cada caractere está associado ao estado de um semáforo (diferente de controlador semafórico, que agrupa os semáforos da intersecção), e não a um movimento veicular. Assim, G , Y e R significam apenas que o farol correspondente está aberto, fechará logo ou está fechado. A indicação r serve para indicar que um semáforo de pedestre está próximo de fechar - na prática, é a situação em que o semáforo de pedestre começa a piscar em vermelho. Além disso, a despeito da indicação fornecida na Figura 18, extraída de (PROJETO..., 2023), nota-se que os semáforos são indicados em cada linha, e não em cada coluna. Na verdade, cada coluna representa uma fase.

	P	S	S	S	P	S	S	S	P	S	S	S								
Intervalo:	1	2	3	4	5	6	7	8	9	10	11	12								
Fase 01	V	A	R	R	R	R	R	R	R	R	R	R								
Fase 02	V	A	R	R	R	R	R	R	R	R	R	R								
Fase 03	V	V	A	A	R	R	R	R	R	R	R	R								
Fase 04	V	V	V	V	V	A	R	R	R	R	R	R								
Fase 05	R	R	R	R	R	R	R	V	V	V	A	R								
Fase 06	R	R	R	V	V	V	V	V	V	V	A	R								
Fase 07	R	R	R	V	V	V	V	V	V	r	r	R								
Fase 08	R	R	R	V	V	V	V	V	V	r	r	R								
Fase 09	V	V	V	V	V	r	R	R	R	R	R	R								
Fase 10	V	A	R	R	R	R	R	R	V	V	V	V								
Fase 11	R	R	R	V	V	r	r	R	R	R	R	R								
Fase 12																				

Figura 18 – Plano com as fases semafóricas.

Plano 01	TC:	150	85	4,0	2,0	2,0	12,0	4,0	2	3,0	27,0	3,0	3	3,0						
Plano 02	TC:	150	85	4,0	2,0	2,0	12,0	4,0	2	3,0	27,0	3,0	3	3,0						
Plano 03	TC:	90	44	4,0	2,0	2,0	11,0	4,0	2	3,0	9,0	3,0	3	3,0						
Plano 04	TC:	150	85	4,0	2,0	2,0	12,0	4,0	2	3,0	27,0	3,0	3	3,0						
Plano 05	TC:	90	44	4,0	2,0	2,0	11,0	4,0	2	3,0	9,0	3,0	3	3,0						
Plano 07	TC:	150	85	4,0	2,0	2,0	12,0	4,0	2	3,0	27,0	3,0	3	3,0						

Figura 19 – Durações das fases semafóricas.

Conforme a Figura 19, há 7 planos semafóricos distintos no que diz respeito ao tempo das fases. O utilizado no projeto é o *plano 2*, correspondente ao horário das 16:00 às 20:00 de segunda-feira até sexta-feira.

Um desafio encontrado durante a calibragem do cenário foi garantir que o fluxo de veículos gerado na simulação passasse pelos pontos de coleta de métricas pelo tempo

mais próximo de 1 hora simulada, sem muitos atrasos. O problema é que em virtude da configuração semafórica, a simulação ultrapassava 2 horas simuladas. Como solução, reduziu-se o tempo de ciclo semafórico de 150 segundos para 75 segundos, mantendo-se a proporção entre as diferentes fases. O problema foi resolvido e com isso foi possível estimar os fluxos.

Por fim, é necessário verificar se a rede está realmente calibrada, o que é feito checando o total de veículos que passaram por um determinado ponto na simulação e comparando-se com os dados reais. Os pontos escolhidos para tal verificação são os pontos de saída da simulação, ou seja, o ponto final de cada uma das rotas. Nota-se que os dados de quantidade de veículos em um dos pontos não são fornecidos diretamente, porém podem ser obtidos facilmente somando o número de veículos que performaram alguns movimentos: M07 + M08 para determinar a saída relativa à Av. Giovanni (sentido leste), M1 foi fornecido diretamente e corresponde à saída da Av. Giovanni (sentido oeste) e M10 também foi fornecido diretamente, sendo relativo à saída da Av. São Paulo. Os dados esperados da simulação são encontrados na Tabela 10.

Rua	Moto	Carro	Ônibus	Caminhão
Av G Leste	734	2372	23	53
Av G Oeste	370	2260	24	69
Av SP Sul	106	477	9	12

Tabela 10 – Dados de fluxos esperados das 17:15 às 18:15

Ao simular a rede calibrada, foram obtidos os resultados presentes em 11. Foi utilizado $T_S = 3600$ segundos.

Rua	Moto	Carro	Ônibus	Caminhão
Av G Leste	734	2375	23	52
Av G Oeste	368	2284	24	67
Av SP Sul	107	473	9	11

Tabela 11 – Dados de fluxo obtidos com simulação no SUMO

Finalmente, a Tabela 12 apresenta os erros de diferenças entre os dados simulados e os dados reais, mostrando a corretude da calibração.

Rua	Moto (%)	Carro (%)	Ônibus (%)	Caminhão (%)
Av G Leste	0.00	0.13	0.00	-1.89
Av G Oeste	-0.54	1.06	0.00	-2.90
Av SP Sul	0.94	-0.84	0.00	-8.33

Tabela 12 – Erros percentuais entre fluxos obtidos via SUMO e fluxos esperados.

Assim, nota-se que o modelo se adequou à realidade, com um erro maior no caso de caminhões, o que ocorre em virtude do baixo número de caminhões (a diferença foi apenas de 1 caminhão - baixo erro absoluto) e com a simulação durando aproximadamente 3800 segundos (próximo de 1 hora). Portanto, é possível considerar o modelo calibrado com sucesso.

5.3.2 Determinação de T_S , tamanho de população e quantidade de gerações

Como a execução das simulações com 3600 segundos se mostrou inviável em termos práticos, foi necessário reduzir o tempo de simulação. Assim, com o objetivo de verificar a deterioração das métricas por conta da diminuição de T_S , foram realizados testes com os seguintes tempos de simulação: 2 minutos, 5 minutos, 10 minutos, 15 minutos, 30 minutos e 1 hora. O objetivo é determinar qual o mínimo $T_S < 3600$ tal que a diferença das métricas 2.2 e 2.5 é pequena ao se reajustar para $T_S = 3600$. Este teste foi realizado sem a presença de paralelismo, dado que o intuito é apenas calcular o tempo de cada simulação individualmente e avaliar a deterioração das métricas, não sendo requerida uma elevada performance. Nota-se que para este teste foram utilizadas populações com 5 indivíduos e 5 gerações, com exceção do caso de 1 hora, o qual foi executado com 4 indivíduos e 3 gerações. A coluna tempo simulado da Tabela 13 corresponde a quanto tempo de tráfego do mundo real foi simulado (T_S), enquanto a segunda coluna corresponde ao tempo total para que o algoritmo genético otimize com base na população e número de gerações definidos. A última coluna é o tempo médio que o SUMO levou para executar cada simulação.

Tempo simulado	Tempo de execução	população	gerações	Tempo médio de simulação
2 minutos	1 minuto e 18 segundos	5	5	3.1 segundos
5 minutos	3 minutos e 4 segundos	5	5	7.3 segundos
10 minutos	6 minutos e 14 segundos	5	5	15.0 segundos
15 minutos	9 minutos e 34 segundos	5	5	23.0 segundos
30 minutos	21 minutos e 30 segundos	5	5	51.6 segundos
1 hora	24 minutos e 30 segundos	4	3	2 minutos e 3 segundos

Tabela 13 – Tempos de execução para diferentes tempos de simulação.

Sobre a deterioração das métricas, para cada solução presente em uma fronteira de Pareto dos casos da tabela anterior, executou-se a simulação com a respectiva política semafórica e com o tempo de simulação aumentado para corresponder a 1 hora (configuração original). A Tabela 14 compara as métricas de cada solução considerando estes dois cenários. Os casos com ∞ são tais que houve *deadlock* ao aplicar a respectiva política semafórica, travando a simulação.

Desta forma, analisando a Tabela 14, é possível concluir que o melhor caso a ser considerado é o de 30 minutos. Isto porque este é o único caso no qual *Deadlocks* não ocorreram ao simular o cenário por 1 hora. Portanto, ao selecionar 30 minutos de

Tempo original	$\bar{st}_{sis}$ original	q_{max} original	$\bar{st}_{sis}$ em 1h	q_{max} em 1h
2 minutos	13.5 segundos	4.6 veículos	inf segundos	inf veículos
2 minutos	11.2 segundos	4.8 veículos	77.3 segundos	15.7 veículos
2 minutos	10.9 segundos	5.1 veículos	74.7 segundos	20.1 veículos
5 minutos	17.4 segundos	6.8 veículos	inf segundos	inf veículos
5 minutos	16.3 segundos	7.2 veículos	80.1 segundos	12.9 veículos
5 minutos	15.9 segundos	8.6 veículos	82.3 segundos	20.4 veículos
10 minutos	35.5 segundos	9.1 veículos	inf segundos	inf veículos
10 minutos	34.5 segundos	10.3 veículos	inf segundos	inf veículos
10 minutos	29.7 segundos	10.4 veículos	40.3 segundos	14.1 veículos
10 minutos	23.0 segundos	11.9 veículos	35.32 segundos	18.8 veículos
15 minutos	51.4 segundos	9.8 veículos	inf segundos	inf veículos
15 minutos	45.5 segundos	10.6 veículos	48.9 segundos	13.2 veículos
15 minutos	43.2 segundos	10.7 veículos	48.4 segundos	12.6 veículos
30 minutos	53.6 segundos	11.7 veículos	56.5 segundos	12.1 veículos
30 minutos	55.4 segundos	11.9 veículos	58.0 segundos	12.8 veículos
30 minutos	60.7 segundos	17.9 veículos	63.5 segundos	19 veículos
30 minutos	66.4 segundos	18 veículos	68.2 segundos	18.3 veículos
1 hora	65.5 segundos	19.3 veículos	65.5 segundos	19.3 veículos
1 hora	68.6 segundos	12.7 veículos	68.6 segundos	12.7 veículos

Tabela 14 – Deterioração das métricas devido ao aumento de tempo de simulação

simulação, é possível reduzir o tempo de execução (computacional) praticamente pela metade e garantir resultados coerentes (sem *Deadlocks*). Além disso, a deterioração das métricas em si foi baixa. Nota-se que o objetivo não era analisar qual tempo de simulação geraria a melhor solução ou algo similar (quando executado com 3600 segundos), até porque se usou uma população a quantidade de gerações baixa. O objetivo foi apenas realizar um teste relativamente rápido para checar o quão baixo poderia ser T_S sem comprometer a otimização considerando que ao final quer-se boa performance para T_S de 1 hora. Por fim, comenta-se que usando a GUI (Graphic User Interface) para acompanhar algumas das simulações cujas métricas deterioraram significativamente, percebeu-se que o maior problema é que com T_S não elevado o bastante não havia tempo suficiente para as filas se formarem. Assim, mesmo que a política semafórica levasse o sistema a um estado de saturação, deteriorando as duas métricas, o baixo tempo de simulação escondia este fato.

5.3.3 Definição dos indivíduos e gerações

Para determinar os parâmetros do NSGA-II, é necessário considerar as limitações do projeto no que diz respeito ao tempo de simulação. A Tabela 13 indica uma média de 51.6 segundos por simulação no caso de simulações com 1800 segundos de tempo simulado, isto sem paralelismo, o que representa um tempo relativamente alto. Considerando que as máquinas nas quais o projeto é executado possuem quatro *cores*, exceto por bordas (quando restarem menos de 4 elementos a serem processados), é possível processar quatro

simulações independentes ao mesmo tempo. Com isso, na prática é como se o tempo médio gasto por cada simulação fosse reduzido em quatro vezes, atingindo 12.9 segundos. Além disso, definiu-se como requisito que o tempo de execução não deveria passar de 6 horas, em média. Assim, tornou-se necessário pesquisar na literatura projetos que utilizam populações pequenas e poucas gerações para se ter como parâmetro proporções válidas.

Neste estudo, foram identificados trabalhos que utilizam 30 indivíduos na população e 50 gerações, o que corresponde a uma quantidade aceitável de simulações para o projeto. Por exemplo, [Du et al. \(2025\)](#) empregam população de 30 indivíduos e 50 gerações no NSGA-II em um framework voltado à otimização energética da alocação de tarefas em sistemas de *shuttles* multinível. De forma semelhante, [Vita \(2009\)](#) também adota essa configuração em experimentos que buscam otimizar o roteamento e o desempenho do tráfego em redes IP por meio de algoritmos genéticos multiobjetivo.

Portanto, decidiu-se a utilização dos parâmetros apresentados em [15](#), de modo a haver 1500 simulações por execução. Com o isso, o tempo de execução foi estimado em 5 horas e 30 minutos, considerado adequado para o projeto.

T_s	População	Gerações
1800 segundos	30 indivíduos	50 gerações

Tabela 15 – Parâmetros considerados

5.3.4 Plano de experimentos

O primeiro passo do procedimento experimental consiste na realização de 10 execuções do algoritmo sobre a rede calibrada de Santo André, com 1800 segundos de execução e populações de 30 indivíduos e 50 gerações. Após este processo, serão obtidas 10 fronteiras de Pareto - as quais fornecem o Tempo Médio de Espera e Tamanho Máximo de Fila - com suas respectivas soluções não dominadas. Além das fronteiras de Pareto, será gerada uma tabela que relaciona os *inputs* (*offsets* e tempos de verde) com os *outputs* (Tempo Médio de Espera, Tamanho Máximo de Fila e Tamanho Médio de Fila).

Em seguida, tornou-se necessário refazer a simulação todas as soluções encontradas com tempo de simulação ampliado para 3600 segundos, haja vista que os resultados obtidos são referentes a 1800 segundos por limitações computacionais, sendo, portanto, inviável sua comparação com o cenário tradicional, o qual foi calibrado para 3600 segundos. Assim, após as executar novamente as simulações, serão obtidas novas fronteiras de Pareto atualizadas e comparáveis ao caso não otimizado. Observa-se que as novas fronteiras de Pareto foram geradas incluindo a solução não otimizada para efeito de avaliação do desempenho individual de cada solução: com qual frequência o algoritmo encontra soluções aceitáveis. Também foi gerada uma tabela idêntica à gerada para as soluções de 1800

segundos. Adicionalmente, foi gerada uma tabela com os ganhos de cada solução em relação ao caso tradicional.

Com todas as soluções finais obtidas, serão geradas duas fronteiras de Pareto finais, uma apenas com as soluções obtidas, com o fim de realizar uma comparação entre as melhores soluções, e outra que inclui a solução corrente, com o fim de verificar a distância entre as obtidas e a corrente.

Nota-se que é possível avaliar qual solução se desempenha melhor apenas se uma não domina a outra: até então, caso isto ocorra, não seria possível performar tal validação. Com este objetivo de comparar as soluções de forma consolidada independente de ser dominada ou não, foi adotada uma métrica escalar de desempenho *Score J*, a qual consiste em uma combinação linear das duas métricas-objetivo normalizadas do NSGA-II: o Tempo Médio de Espera Normalizado ($\bar{st}_{sis}$) e o Tamanho Máximo de Fila Normalizado (q_{max}). Foram atribuídos pesos iguais a ambas as métricas. A fórmula 5.1 representa tal métrica.

$$J = 0.5 \cdot \bar{st}_{sis} + 0.5 \cdot q_{max}, \quad (5.1)$$

Duas novas tabelas foram geradas com esta métrica: uma de dados brutos, com a solução corrente inclusa, e outra de ganhos relativos, cuja fórmula é dada em 5.2.

$$G(\%) = \frac{M_{corr} - M_{solução}}{M_{corr}} \times 100, \quad (5.2)$$

em que M_{corr} é a métrica da solução tradicional e $M_{solução}$ é a métrica da solução otimizada. Esta é uma fórmula genérica que é aplicada às métricas objetivo e Tamanho Médio de Fila. A fórmula de normalização das métricas para cálculo do *Score J* é a seguinte:

$$x_{scaled} = \frac{x - x_{min}}{x_{max} - x_{min}}, \quad (5.3)$$

em que x representa o valor original da métrica a ser normalizada, x_{min} corresponde ao menor valor observado dessa métrica entre todas as soluções analisadas e x_{max} corresponde ao maior valor observado.

Com estes passos finalizados e os resultados obtidos, são realizadas discussões acerca do desempenho geral do algoritmo - envolvendo todas as execuções - e acerca de sua eficiência - se o mesmo encontra boas soluções com frequência. Também podem ser discutidos outros aspectos observados nos resultados, porém os dois anteriores consistem nos principais apontamentos. A Figura 20 resume o plano de experimentos.

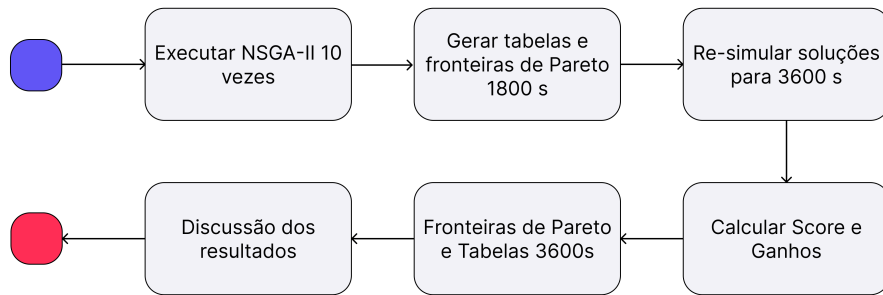


Figura 20 – Resumo do Setup Experimental

5.4 Resultados experimentais

Nesta seção, são apresentados os resultados obtidos relativos à otimização performeda pelo algoritmo multiobjetivo NSGA2 nos semáforos localizados no cruzamento entre as avenidas Giovanni Battista e São Paulo, na cidade de Santo André - SP. Ademais, também é feita uma discussão sobre os resultados obtidos, comparando-os com a política semafórica corrente.

5.4.1 Resultado das Execuções de 1800 segundos

Após o término das 10 execuções do programa, foram obtidas as fronteiras de Pareto individuais com as soluções não dominadas. As Figuras a seguir (21 22 23 24 25) representam as mesmas.

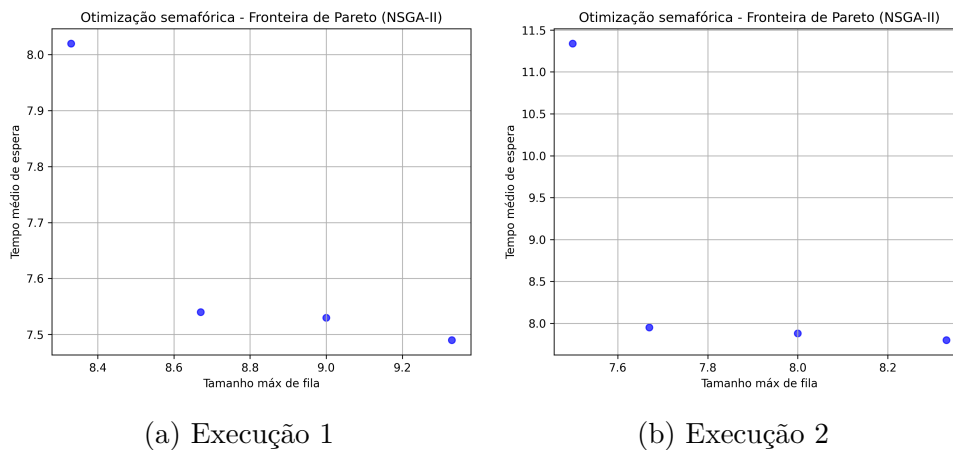


Figura 21 – Fronteiras de Pareto 1800s — Execuções 1 e 2

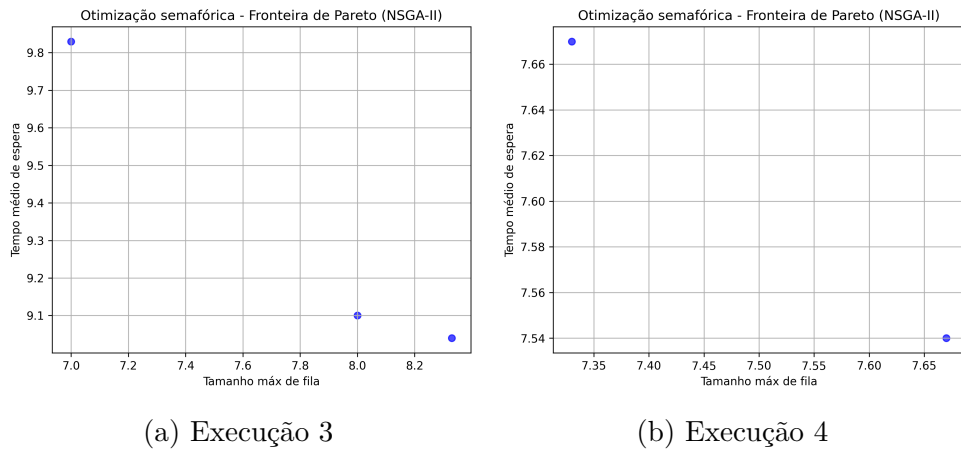


Figura 22 – Fronteiras de Pareto 1800s — Execuções 3 e 4

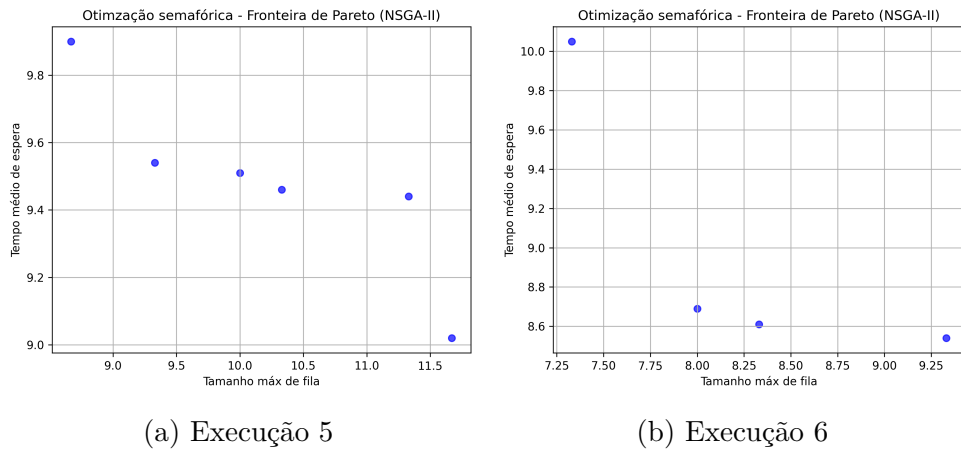


Figura 23 – Fronteiras de Pareto 1800s — Execuções 5 e 6

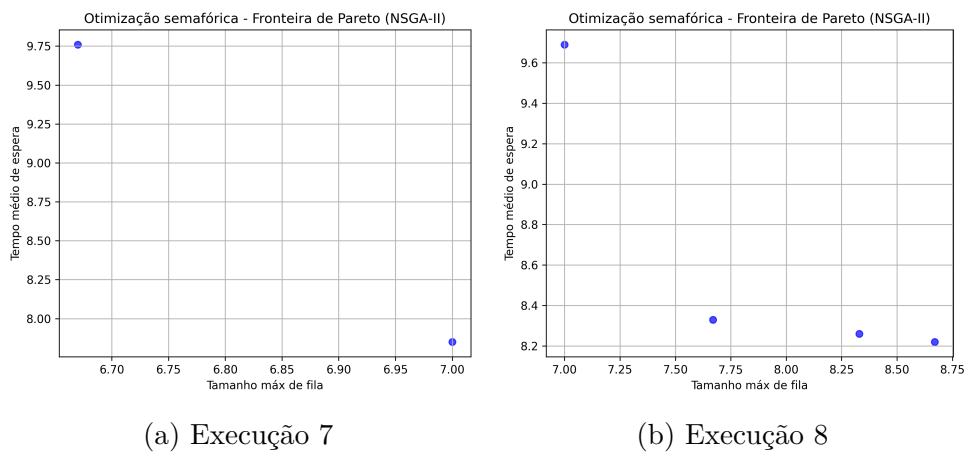
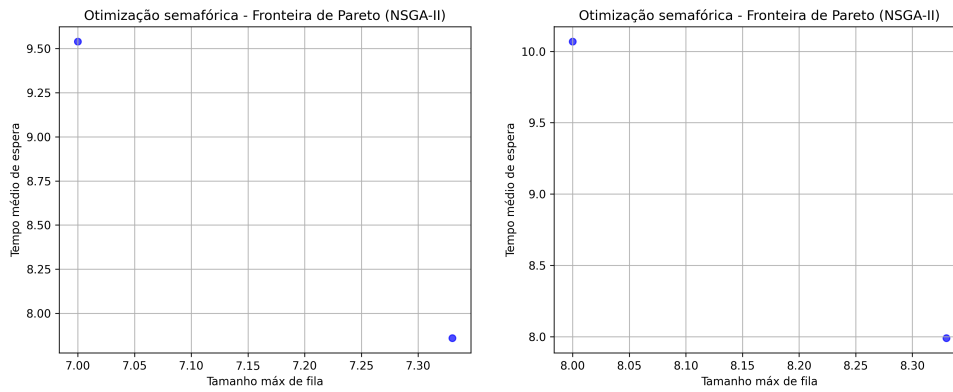


Figura 24 – Fronteiras de Pareto 1800s — Execuções 7 e 8

A Tabela 16 indica os *inputs* e *outputs* das execuções. O primeiro consiste em *offsets* semafóricos, os tempos de verde determinados pelo NSGA II e o tempo de verde do último semáforo. O segundo consiste nos tempos médios de viagem, tamanho médio de fila e tamanho máximo de fila.



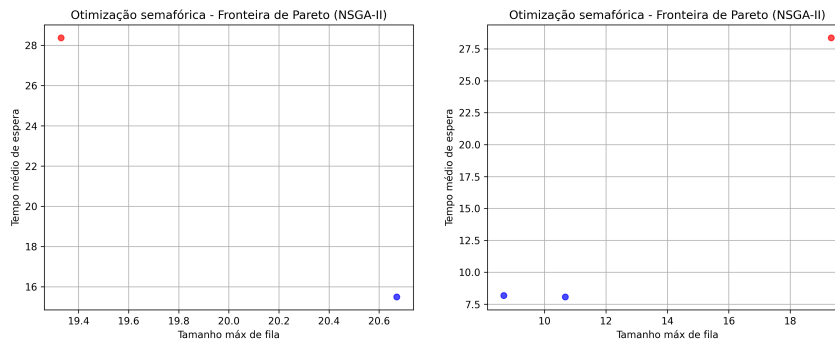
(a) Execução 9

(b) Execução 10

Figura 25 – Fronteiras de Pareto 1800s — Execuções 9 e 10

5.4.2 Resultado das execuções de 3600 segundos

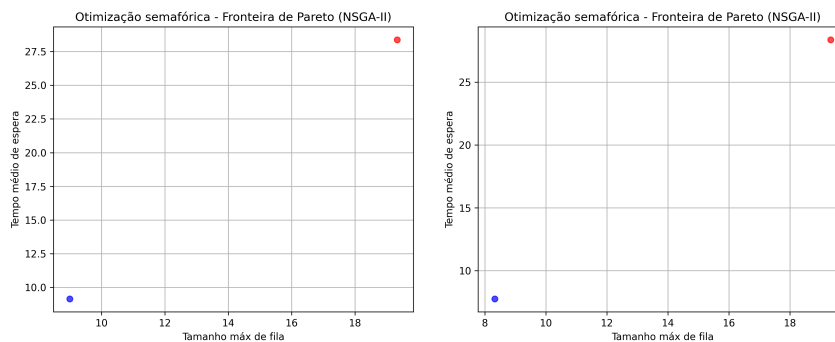
Após a realizar novamente a simulação das soluções não dominadas de 1800 segundos, mas em 3600 segundos, foram obtidas novas fronteiras de Pareto. As Figuras a seguir (26 27 28 29 30) mostram estes resultados comparados com a solução tradicional.



(a) Execução 1

(b) Execução 2

Figura 26 – Fronteiras de Pareto 3600s — Execuções 1 e 2



(a) Execução 3

(b) Execução 4

Figura 27 – Fronteiras de Pareto 3600s — Execuções 3 e 4

Exec	Offsets	Greens	Last Greens	Avg Wait	Avg Queue	Max Queue
1	69.52, 52.61, 50.79	56.80, 46.18, 1.27, 54.09	3.70, 19.55, 14.91	8.02	7.40	8.33
1	73.18, 52.61, 50.71	59.82, 45.67, 1.21, 54.09	0.68, 20.12, 14.91	7.54	7.11	8.67
1	73.40, 52.61, 48.83	56.81, 46.18, 1.26, 54.09	3.69, 19.56, 14.91	7.49	7.07	9.33
1	72.92, 52.60, 48.83	56.81, 46.18, 0.34, 54.09	3.69, 20.49, 14.91	7.53	6.98	9.00
2	38.74, 15.23, 13.28	12.23, 50.08, 0.93, 52.58	48.27, 15.98, 16.42	11.34	10.87	7.50
2	34.90, 15.23, 13.27	12.24, 46.75, 0.93, 52.58	48.26, 19.32, 16.42	7.88	7.48	8.00
2	26.31, 15.23, 13.18	12.24, 46.66, 0.93, 52.58	48.26, 19.41, 16.42	7.80	7.48	8.33
2	25.02, 15.23, 13.18	12.24, 45.51, 0.93, 52.58	48.26, 20.56, 16.42	7.95	7.61	7.67
3	61.29, 56.03, 58.59	25.86, 48.37, 1.15, 50.31	34.64, 17.48, 18.69	9.83	9.29	7.00
3	61.30, 56.07, 58.78	25.86, 45.58, 1.79, 51.32	34.64, 19.63, 17.68	9.04	8.28	8.33
3	61.32, 56.03, 56.61	25.85, 48.25, 1.77, 50.30	34.65, 16.98, 18.70	9.10	8.57	8.00
4	37.59, 25.28, 20.57	17.84, 47.36, 0.56, 54.37	42.66, 19.09, 14.63	7.67	7.34	7.33
4	36.70, 25.28, 20.57	10.45, 47.36, 0.56, 53.66	50.05, 19.09, 15.34	7.54	7.17	7.67
5	4.79, 69.19, 67.22	15.96, 42.10, 3.69, 51.63	44.54, 21.21, 17.37	9.44	8.32	11.33
5	4.70, 57.71, 67.20	15.96, 43.68, 3.62, 51.63	44.54, 19.70, 17.37	9.90	8.78	8.67
5	4.54, 57.70, 61.46	15.96, 43.65, 3.62, 51.63	44.54, 19.72, 17.37	9.51	8.53	10.00
5	4.79, 68.77, 67.20	15.96, 45.12, 3.68, 51.63	44.54, 18.20, 17.37	9.46	8.62	10.33
5	4.78, 58.15, 61.47	15.96, 42.19, 3.63, 51.62	44.54, 21.18, 17.38	9.54	8.46	9.33
5	4.80, 67.16, 67.27	15.96, 42.19, 3.63, 51.85	44.54, 21.18, 17.15	9.02	8.05	11.67
6	37.00, 28.37, 31.91	57.75, 43.93, 4.14, 53.82	2.75, 18.92, 15.18	8.61	8.00	8.33
6	36.94, 29.87, 25.95	57.75, 43.93, 0.28, 50.92	2.75, 22.79, 18.08	8.69	7.96	8.00
6	37.02, 28.21, 32.13	57.75, 41.73, 4.16, 53.81	2.75, 21.11, 15.19	8.54	7.71	9.33
6	36.98, 28.37, 31.91	57.75, 45.63, 4.14, 51.27	2.75, 17.22, 17.73	10.05	9.41	7.33
7	26.35, 24.40, 20.05	26.45, 46.94, 0.52, 52.17	34.05, 19.54, 16.83	7.85	7.46	7.00
7	26.35, 24.41, 20.05	26.44, 50.22, 0.52, 52.09	34.06, 16.26, 16.91	9.76	9.52	6.67
8	38.52, 47.35, 42.58	30.64, 50.09, 0.60, 51.77	29.86, 16.30, 17.23	9.69	9.25	7.33
8	39.21, 47.34, 42.58	31.36, 47.91, 0.63, 51.76	29.14, 18.46, 17.24	8.26	7.73	8.33
8	39.25, 47.34, 42.58	30.58, 47.91, 0.63, 53.32	29.92, 18.46, 15.68	8.33	7.85	7.67
8	39.22, 42.03, 42.58	30.51, 48.01, 0.60, 51.76	29.99, 18.39, 17.24	8.22	7.75	8.67
9	32.57, 33.70, 29.90	24.35, 47.09, 0.48, 54.74	36.15, 19.43, 14.26	7.86	7.58	7.33
9	32.85, 29.07, 30.05	24.27, 46.95, 3.32, 55.78	36.23, 16.73, 13.22	9.54	9.20	7.00
10	5.30, 56.08, 59.79	5.41, 45.97, 3.81, 52.98	55.09, 17.22, 16.02	10.07	9.37	8.00
10	5.25, 56.08, 52.95	5.41, 45.97, 3.81, 52.99	55.09, 17.22, 16.01	7.99	7.38	8.33

Tabela 16 – Resultados das Execuções 1–10, 1800s

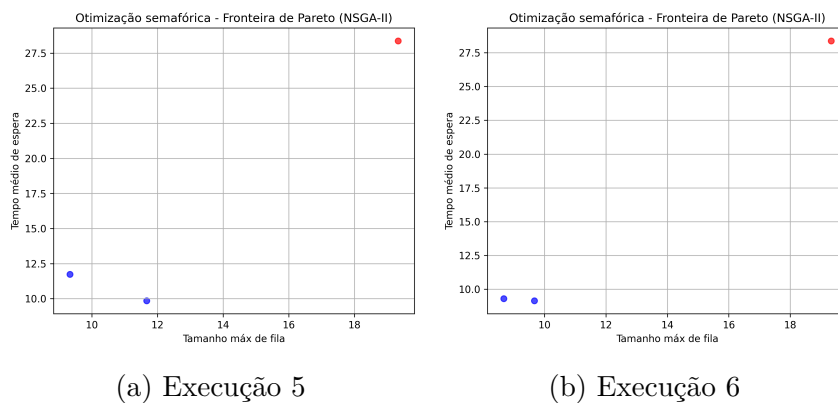


Figura 28 – Fronteiras de Pareto 3600s — Execuções 5 e 6

A Tabela 17 a seguir relaciona os *inputs* com os novos resultados. Nota-se que apenas as soluções não dominadas estão presentes.

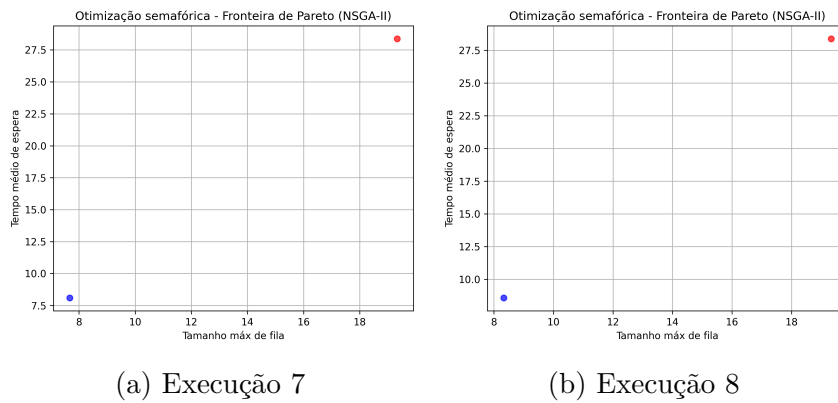


Figura 29 – Fronteiras de Pareto 3600s — Execuções 7 e 8

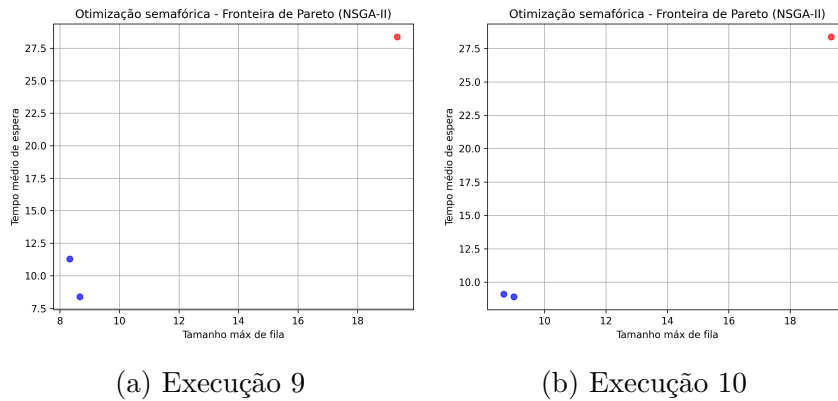


Figura 30 – Fronteiras de Pareto 3600s — Execuções 9 e 10

Exec	Offsets	Greens	Last Greens	Avg Wait	Avg Queue	Max Queue
1	69.52, 52.61, 50.79	56.80, 46.18, 1.27, 54.09	3.70, 19.55, 14.91	15.50	11.90	20.67
2	26.31, 15.23, 13.18	12.24, 46.66, 0.93, 52.58	48.26, 19.41, 16.42	8.19	7.79	8.67
2	25.02, 15.23, 13.18	12.24, 45.51, 0.93, 52.58	48.26, 20.56, 16.42	8.07	7.57	10.67
3	61.30, 56.07, 58.78	25.86, 45.58, 1.79, 51.32	34.64, 19.63, 17.68	9.15	8.31	9.00
4	36.70, 25.28, 20.57	10.45, 47.36, 0.56, 53.66	50.05, 19.09, 15.34	7.76	7.35	8.33
5	4.79, 68.77, 67.20	15.96, 45.12, 3.68, 51.63	44.54, 18.20, 17.37	9.85	8.94	11.67
5	4.78, 58.15, 61.47	15.96, 42.19, 3.63, 51.62	44.54, 21.18, 17.38	11.73	10.22	9.33
6	37.00, 28.37, 31.91	57.75, 43.93, 4.14, 53.82	2.75, 18.92, 15.18	9.15	8.39	9.67
6	36.94, 29.87, 25.95	57.75, 43.93, 0.28, 50.92	2.75, 22.79, 18.08	9.30	8.38	8.67
7	26.35, 24.40, 20.05	26.45, 46.94, 0.52, 52.17	34.05, 19.54, 16.83	8.09	7.66	7.67
8	39.21, 47.34, 42.58	31.36, 47.91, 0.63, 51.76	29.14, 18.46, 17.24	8.57	8.00	8.33
9	32.57, 33.70, 29.90	24.35, 47.09, 0.48, 54.74	36.15, 19.43, 14.26	8.38	8.01	8.67
9	32.85, 29.07, 30.05	24.27, 46.95, 3.32, 55.78	36.23, 16.73, 13.22	11.29	10.78	8.33
10	5.30, 56.08, 59.79	5.41, 45.97, 3.81, 52.98	55.09, 17.22, 16.02	9.09	8.42	8.67
10	5.25, 56.08, 52.95	5.41, 45.97, 3.81, 52.99	55.09, 17.22, 16.01	8.90	8.19	9.00

Tabela 17 – Resultados das Execuções 1–10, 3600s

5.4.3 Fronteira de Pareto final

Com os resultados das fronteiras de Pareto individuais, foi gerada a fronteira de Pareto final, a qual consiste na Figura 32. Também foi gerada a fronteira de Pareto comparando as soluções não dominadas e a solução corrente, mostrada na Figura 31.

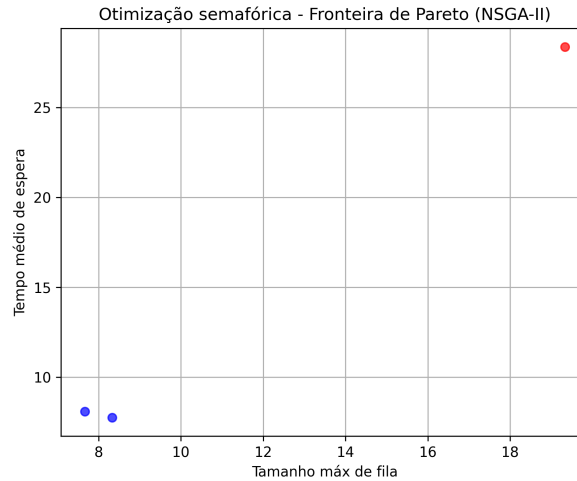


Figura 31 – Fronteira de Pareto final vs Solução Corrente

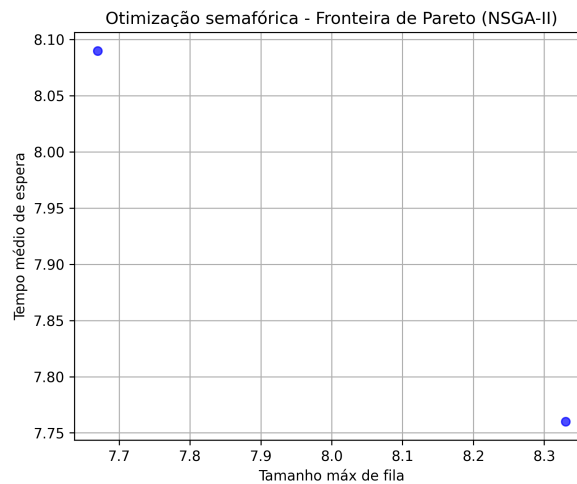


Figura 32 – Fronteira de Pareto final, sem solução corrente

A Tabela 18 resume os *inputs* das soluções ótimas e os relaciona com os *outputs*, os quais foram mencionados anteriormente.

Exec	Offsets	Greens	Last Greens	Avg Wait	Avg Queue	Max Queue
4	36.70, 25.28, 20.57	10.45, 47.36, 0.56, 53.66	50.05, 19.09, 15.34	7.76	7.35	8.33
7	26.35, 24.40, 20.05	26.45, 46.94, 0.52, 52.17	34.05, 19.54, 16.83	8.09	7.66	7.67

Tabela 18 – Soluções da fronteira de Pareto Final

Por fim, a Tabela 19 demonstra os resultados numéricos das soluções ótimas encontradas em comparação com o cenário tradicional. Relembrando sua fórmula, nota-se

que a métrica Avg. Queue não participa da composição de J , já que não foi utilizada como critério de otimização no NSGA-II. Ela é mantida nas tabelas apenas como informação complementar, permitindo uma análise mais ampla do comportamento geral do tráfego.

Execução	Avg Wait Time	Max Queue	Avg Queue	Score [J]
Tradicional	28.38	19.3333	21.34	1
4	7.76	8.33	7.35	0.03
7	8.09	7.67	7.66	0.005

Tabela 19 – Comparação entre o caso tradicional e soluções da fronteira de Pareto

5.5 Discussão dos resultados

5.5.1 Comparação dos tipos de execução

O primeiro ponto a ser discutido é a alteração da fronteira de Pareto das simulações de 1800 segundos quando executadas por 3600 segundos. Conforme o estudo feito para validar a presença de degradação no resultado (o qual foi explicado anteriormente no presente documento), os resultados das simulações de 1800 segundos não apresentaram degradações consideráveis em relação às de 3600 segundos. Entretanto, as soluções de Pareto encontradas para o primeiro possuem resultados bem próximos, dentro de uma mesma execução. Desta forma, apesar de não haver degradação, os resultados alteram-se um pouco ao ponto de, na simulação de 3600 segundos, uma solução que não era dominada passar a ser dominada por alguma outra. Em virtude disto, é possível verificar uma redução do número de soluções nas fronteiras de Pareto de cada execução, quando simuladas novamente para 1 hora. A exemplo, é possível observar este comportamento nas imagens [21a](#) e [26a](#), havendo redução de 3 pontos, [21b](#) e [26b](#), havendo redução de 2 pontos, [22a](#) e [27a](#), havendo redução de 2 pontos, [22b](#) e [27b](#), havendo redução de 1 ponto, [23a](#) e [28a](#), havendo redução de 4 pontos, [23b](#) e [28b](#), havendo redução de 2 pontos, [24a](#) e [29a](#), havendo redução de 1 ponto e [24b](#) e [29b](#), havendo redução de 3 pontos.

Este resultado não é desejado em situações que deseja-se buscar a melhor solução possível, visto que talvez houvesse alguma solução desconsiderada inicialmente nas simulações de 1800 segundos que gerasse uma solução melhor que as demais para 3600 segundos. Entretanto, no projeto atual, os resultados não foram prejudicados, dado que bastava encontrar uma solução melhor e com certa frequência, dadas as configurações do projeto.

5.5.2 Desempenho conjunto das execuções

O segundo ponto a ser discutido é relativo ao desempenho das 10 execuções em conjunto. A tabela a seguir [20](#) foi construída com base na Tabela 19, e indica o ganho de desempenho em relação à solução tradicional. Nota-se que ambas as soluções dominam-na

significativamente, visto que possuem ganhos superiores a 50% em todas as métricas. Outra prova da disparidade entre as soluções é a Figura 31, na qual é possível ver a distância entre os pontos solução e a solução tradicional, além de ser possível identificar que esta é dominada pelas demais.

Execução	Avg Wait (%)	Max Queue (%)	Avg Queue (%)
4	72.6%	56.9%	65.6%
7	71.5%	60.3%	64.1%

Tabela 20 – Ganho percentual de desempenho da solução geral em relação ao caso tradicional

Este resultado possibilita a conclusão de que o software implementado é capaz de otimizar de maneira significativa os tempos semafóricos de uma via no simulador, caso a mesma não esteja otimizada. Entretanto, nota-se que ainda não foi provado que encontrará uma boa solução para todas as execuções com os parâmetros do NSGAI configurados. Para tal, é necessário verificar as soluções de cada execução individualmente.

5.5.3 Desempenho individual de execução

Por fim, o último ponto a ser discutido é relativo ao desempenho individual de cada uma das soluções. Nota-se nas Figuras 26, 27, 28, 29 e 30 que praticamente todas as soluções de todas as execuções dominam o caso corrente. Há apenas uma exceção, a qual é relativa à primeira execução e evidenciada pela Figura 26a. A Tabela 21 mostra as métricas resultado das melhores soluções de cada execução e, ao analisar J nesta tabela, é possível verificar o fato de todas soluções serem significativamente melhores que a tradicional, exceto a 1.1, a qual, apesar de ser melhor, possui um *Score J* mais próximo. A Tabela 22 possibilita uma melhor análise dos ganhos individuais de cada métrica, inclusive da solução 1.1: enquanto o ganho em relação à métrica *avg wait time* é significativo, o ganho da métrica *max queue length* é negativo, indicando que a solução corrente é melhor neste aspecto, o que já poderia ser concluído analisando a fronteira de Pareto 26a. Entretanto, a métrica J indica que a solução encontrada em 1.1 é melhor: o valor para ela é 0.688, enquanto a solução referente à política corrente obteve 0.949.

Portanto, é possível concluir que a probabilidade de encontrar uma solução melhor que a corrente, para este problema, não é baixa, apesar de haver chances obter-se soluções ruins, conforme mostra a solução 1.1. Assim, é essencial que o programa seja executado múltiplas vezes para garantir uma solução boa para o problema e que a solução seja previamente testada no simulador e seu desempenho previamente analisado antes de implementá-la na prática.

Execução	Avg Wait Time	Max Queue	Avg Queue	Score [J]
Tradicional	28.38	19.3333	21.34	0.949
1.1	15.50	20.67	11.90	0.688
2.1	8.19	8.67	7.79	0.049
2.2	8.07	10.67	7.57	0.123
3.1	9.15	9.00	8.31	0.085
4.1	7.76	8.33	7.35	0.025
5.1	9.85	11.67	8.94	0.205
5.2	11.73	9.33	10.22	0.160
6.1	9.15	9.67	8.39	0.111
6.2	9.30	8.67	8.38	0.076
7.1	8.09	7.67	7.66	0.008
8.1	8.57	8.33	8.00	0.045
9.1	8.38	8.67	8.01	0.053
9.2	11.29	8.33	10.78	0.111
10.1	9.09	8.67	8.42	0.071
10.2	8.90	9.00	8.19	0.079

Tabela 21 – Comparação entre o caso tradicional e soluções da fronteira de Pareto

Execução	Ganho Avg Wait (%)	Ganho Max Queue (%)	Ganho Avg Queue (%)
1.1	45.4%	-6.9%	44.3%
2.1	71.1%	55.2%	63.5%
2.2	71.6%	44.8%	64.5%
3.1	67.8%	53.4%	61.1%
4.1	72.6%	56.9%	65.6%
5.1	65.3%	39.6%	58.1%
5.2	58.7%	51.7%	52.1%
6.1	67.8%	50.0%	60.7%
6.2	67.2%	55.2%	60.7%
7.1	71.5%	60.3%	64.1%
8.1	69.8%	56.9%	62.5%
9.1	70.5%	55.2%	62.5%
9.2	60.2%	56.9%	49.5%
10.1	68.0%	55.2%	60.5%
10.2	68.7%	53.5%	61.6%

Tabela 22 – Ganho percentual de desempenho das soluções individuais em relação ao cenário tradicional

5.5.4 Comparação visual

Outra maneira de avaliar a eficiência das soluções é inspecionar estas simulações visualmente, mesmo que não sejam a maneira mais eficiente. Para tal exercício, foram consideradas as duas melhores soluções encontradas, 4 (Figura 34) e 7 (Figura 35), além do cenário tradicional (Figura 33). As capturas foram tiradas na iminência de abertura dos semáforos da avenida Giovanni Battista (na horizontal) por volta do instante 2200s de simulação.

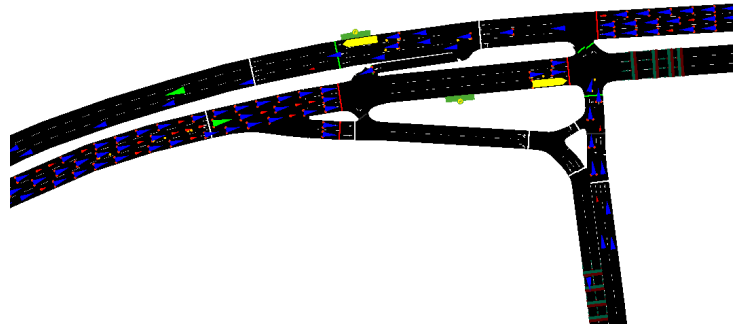


Figura 33 – Captura de tela com trânsito em cenário com política semafórica corrente.

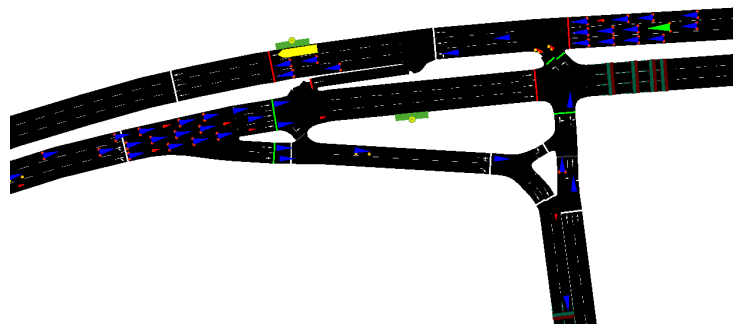


Figura 34 – Captura de tela com trânsito em cenário com política semafórica do caso 4.2.

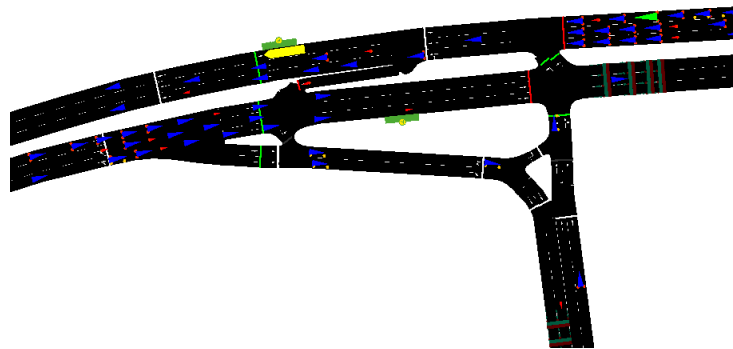


Figura 35 – Captura de tela do caso 7.1

Entre os casos 4 e 7 dificilmente nota-se diferenças na formação de filas, dado que os casos são semelhantes. A comparação se mostra efetiva entre os casos tradicionais e os dois otimizados. Nota-se que a fila se estende por toda a via considerada no caso tradicional em ambos os sentidos (leste e oeste), enquanto isto não ocorre para os outros casos. Isto mostra que ela cresce de maneira mais rápida para o primeiro cenário, o que resulta em um trânsito maior. Já para as simulações otimizadas, o pequeno tamanho de fila demonstra um crescimento pequeno ou até inexistente, comprovando eficiência do resultado encontrado pelo NSGA-II.

6 Considerações Finais

6.1 Conclusões

A partir dos experimentos realizados, observou-se que o sistema proposto foi capaz de otimizar os tempos semaforicos de forma consistente, apesar das limitações computacionais impostas pelo projeto. Como o tempo de simulação de 3600 segundos gerava execuções longas, as mesmas foram configuradas para executar por apenas 1800 s, população de 30 indivíduos e 50 gerações, o que reduzindo o tempo de execução, mas também restringindo a eficácia do algoritmo. Ainda assim, o algoritmo NSGA-II conseguiu encontrar soluções capazes de reduzir significativamente tanto o tempo médio de espera quanto o tamanho das filas, conforme mostrado na seção de resultados.

Apesar dos resultados positivos, verificou-se que uma das execuções apresentou solução com desempenho ligeiramente semelhante ao caso tradicional quando avaliada em tempo de 3600 segundos. Esse comportamento não invalida o que foi desenvolvido, mas evidencia o impacto das limitações de tempo e capacidade de execução. Com um número reduzido de gerações e população limitada, é possível que o algoritmo não convirja para soluções boas o suficiente para superar o caso real. Outro problema possível é a deterioração de resultado ocasionada pela redução do tempo de simulação que, apesar de não ter ocorrido nos testes de deterioração, é completamente plausível.

Dessa forma, recomenda-se que, em aplicações práticas, o algoritmo seja executado mais vezes, com população e número de gerações maior, além de simulações completas desde o início. Com isso, é esperado que as soluções obtidas sejam ainda mais consistentes e representativas em relação ao comportamento real do sistema. Mesmo sob tais restrições, os resultados indicam que a abordagem baseada em algoritmos evolutivos e simulação de tráfego é eficaz e promissora para a otimização de tempos semaforicos em interseções urbanas.

6.2 Contribuições

Este trabalho apresentou um estudo sobre a otimização de tempos semaforicos utilizando o algoritmo genético multiobjetivo NSGA-II integrado com um modelo de simulação (SUMO). O objetivo do trabalho foi alcançado, visto que todas as 10 execuções obtiveram resultados superiores ao cenário real (mesmo havendo uma execução cuja solução não domina a solução corrente, de acordo com o Score J , ela a supera), demonstrando que algoritmos genéticos podem ser utilizados como método de otimização de tempos

semafóricos com poucas gerações e populações pequenas. Além disso, o *software* foi desenvolvido para solucionar qualquer rede viária com semáforos configurados no simulador SUMO, sendo teoricamente, portanto, possível aplicá-lo em intersecções mais complexas, apesar de ser recomendado um novo estudo de eficiência do algoritmo neste caso. Por fim, o sistema foi codificado de forma que a implementação de outros algoritmos genéticos fosse facilitada, bastando alterar uma chamada de função no código fonte.

6.3 Trabalhos futuros

Este item apresenta recomendações para a continuidade do trabalho, fundamentadas nos resultados obtidos e nas limitações identificadas ao longo do desenvolvimento do projeto.

1. **Aumentar o tempo de simulação** Como visto nos tópicos anteriores, as simulações precisaram ser reduzidas de 3600 segundos para 1800 segundos devido a limitações computacionais e tempo hábil para conclusão do projeto. Portanto, uma das evoluções futuras para este projeto é a execução das simulações com tempo total e em máquinas mais potentes com o fim de reduzir o tempo de execução e aumentar sua eficácia.
2. **Aumentar parâmetros do NSGA II** Outra melhoria possível é executar o algoritmo com parâmetros maiores, como exemplo a população e número de gerações. Apesar de aumentar o número de simulações no SUMO, tal modificação reduz o risco de ocorrer convergência para soluções sub-ótimas.
3. **Utilização de outros ou mais métricas objetivo** A alteração da configuração das métricas, seja quantitativamente ou qualitativamente, para efeito de comparação com o método atual. Verificar se a eficiência do algoritmo cresce significativamente frente ao aumento de complexidade do sistema. A exemplo de novas métricas, velocidade média atual em relação a velocidade média de via livre: aumenta a complexidade de coleta de métricas e de inserção como métrica objetivo no NSGA II, mas se bem aplicada, talvez mais efetiva que tempo de espera.
4. **Resolução com outros algoritmos evolutivos** A implementação de outros softwares que resolvem o mesmo problema, porém com outros algoritmos evolutivos (diferentes do NSGA II). Além disso, realizar um balanço de melhoria / perda de desempenho e aumento de eficiência na busca por uma solução boa.
5. **Otimização de outros parâmetros semafóricos** Neste projeto, são definidos tempos de verde e offsets, não alterando outros aspectos dos semáforos, como período e criação / remoção de fases. Um ponto interessante é abordar tais tópicos como foco das otimizações e comparar com o modelo implementado neste projeto. Além disso, outra recomendação é a união da otimização com base nos tempos de verde e offsets com a melhoria em outros parâmetros.

6. **Aplicação prática** Por fim, a implementação dos resultados obtidos neste projeto em um cruzamento real com o fim de validar se o simulador é fidedigno com a realidade. A princípio, é recomendado implementar seus resultados com caráter experimental, em semáforos não tão críticos para, em seguida, implementá-los em pontos de gargalo em uma rede viária.

Referências

BLANK, J.; DEB, K. pymoo: Multi-objective optimization in python. *IEEE Access*, v. 8, p. 89497–89509, 2020. Disponível em: <<https://ieeexplore.ieee.org/document/9094256>>. Citado na página 53.

COELLO, C. A. C.; LAMONT, G. B.; VELDHUIZEN, D. A. V. *Evolutionary Algorithms for Solving Multi-Objective Problems*. 2nd. ed. New York: Springer, 2007. (Genetic and Evolutionary Computation Series). Citado na página 33.

DEB, K. et al. A fast and elitist multiobjective genetic algorithm: Nsga-ii. *IEEE Transactions on Evolutionary Computation*, v. 6, n. 2, p. 182–197, abr. 2002. Citado 2 vezes nas páginas 11 e 41.

DLR – German Aerospace Center. *TraCI: Traffic Control Interface for SUMO*. 2025. Parte da distribuição oficial do simulador SUMO. Acesso em: 04/12/2025. Disponível em: <<https://pypi.org/project/traci/>>. Citado na página 53.

DU, P. et al. Hybrid mcmf–nsga-ii framework for energy-aware task assignment in multi-tier shuttle systems. *Applied Sciences*, v. 15, n. 5, p. 1234–1250, 2025. Citado na página 62.

LEAL, S. S.; ALMEIDA, P. Traffic light optimization using non-dominated sorting genetic algorithm (nsga2). *Scientific Reports*, vol. 13, no. 1, 2023. Citado na página 26.

LOPEZ, P. A. et al. Microscopic traffic simulation using sumo. *IEEE Intelligent Transportation Systems Conference (ITSC)*, p. 2575–2582, 2018. Disponível em: <<https://ieeexplore.ieee.org/abstract/document/8569938/>>. Citado na página 13.

PROJETO da Disciplina PTR 3514 – Sistemas Inteligentes de Transporte. 2023. Material da disciplina PTR3514, fornecido pelo Professor Dr. Claudio Luiz Marte. Citado 3 vezes nas páginas 54, 55 e 58.

PTV PLANUNG TRANSPORT VERKEHR GMBH. *PTV VISSIM 2025 User Manual*. Karlsruhe, Germany, 2025. Citado na página 42.

TEO, K. T. K.; KOW, W. Y.; CHIN, Y. K. Optimization of traffic flow within an urban traffic light intersection with genetic algorithm. *Second International Conference on Computational Intelligence, Modelling and Simulation, set. 2010.*, 2010. Citado na página 26.

VITA, S. S. B. *Algoritmos Genéticos Multiobjetivos Aplicados à Engenharia de Tráfego em Redes IP*. Dissertação (Dissertação de Mestrado) — Universidade Federal de Uberlândia, Uberlândia, MG, 2009. Citado na página 62.

ZHAO HONGXING, e. a. Multi-objective optimization of traffic signal timing using non-dominated sorting artificial bee colony algorithm for unsaturated intersections. *Archives of Transport*, vol. 46, no. 2, 2018. Citado na página 26.

Apêndices

APÊNDICE A – Detalhamento do *non-dominated sorting* do NSGA-II

Como apresentado em 2.3 e em 2.4, o algoritmo usado para implementar o *non-dominated sorting* no NSGA-II melhora a complexidade assintótica de $O(M \cdot N^3)$ para $O(M \cdot N^2)$. Neste apêndice, detalha-se a lógica usada na implementação eficiente.

Para se fazer o processo em $O(M \cdot N^2)$, primeiro é realizado um pré-processamento. São feitas todas as comparações dois a dois, o que é da ordem de $O(M \cdot N^2)$, e guardam-se as seguintes informações: n_{x_i} , a quantidade de soluções que dominam a solução x_i , e S_{x_i} , o conjunto de soluções que a solução x_i domina. Observe que os elementos de F_1 são exatamente aqueles tais que $n_{x_i} = 0$. Sabendo-se tais informações, inicia-se um *loop*. A cada iteração, passa-se pelos elementos da população que ainda não foram alocados em alguma fronteira. Caso sua contagem n_{x_i} seja 0, aloca-se o elemento à fronteira correspondente ao número da iteração (i.e primeira iteração implica que elemento pertence a F_1 , e assim por diante). Em seguida, passa-se por todos os elementos x_j tais que $x_i \prec x_j$ e subtrai-se 1 de n_{x_j} . A partir do pseudo-código 3, explica-se de maneira mais precisa o funcionamento do algoritmo.

Algorithm 3 Fast Non-Dominated Sorting (NSGA-II)**Require:** População P com N indivíduos, cada um com M objetivos.**Ensure:** Conjunto de frentes de Pareto $\mathcal{F} = \{F_1, F_2, \dots\}$

▷ — Inicialização —

```

1: for indivíduo  $i$  da população do
2:    $contagem\_dominadores[i] \leftarrow 0$ 
3:    $S[i] \leftarrow \emptyset$                                 ▷ Indivíduos dominados por  $i$ 
4: end for
                                     ▷ — Pré-processamento: único laço percorrendo pares  $(i, j)$  —
5: for  $(i, j)$  em todos os pares da população do
6:   if  $i \prec j$  then
7:      $contagem\_dominadores[j] \leftarrow contagem\_dominadores[j] + 1$ 
8:     Adicionar  $j$  em  $S[i]$ 
9:   end if
10: end for
                                     ▷ — Construção de todas as frentes (inclusive a primeira) —
11:  $\mathcal{F} \leftarrow \emptyset$ 
12:  $k \leftarrow 1$ 
13: while existe algum indivíduo não processado do
14:    $F_k \leftarrow \emptyset$ 
                                     ▷ Seleciona todos com dominated_count = 0
15:   for indivíduo  $i$  da população não processado do
16:     if  $contagem\_dominado[i] = 0$  then
17:       Adicionar  $i$  em  $F_k$ 
18:     end if
19:   end for
                                     ▷ Reduz contagens para a próxima frente
20:   for cada  $i \in F_k$  do
21:     for cada  $j \in S[i]$  do
22:        $contagem\_dominado[j] \leftarrow contagem\_dominado[j] - 1$ 
23:     end for
24:   end for
25:   Adicionar  $F_k$  a  $\mathcal{F}$ 
26:    $k \leftarrow k + 1$ 
27: end while

```

O código presente em 3 é dividido em duas grandes partes: o pré-processamento e a determinação das frentes de Pareto, que vai da linha 11 em diante. Primeiro, aborda-se o pré-processamento. É feito um *loop* considerando todas as comparações dois a dois de elementos da população. Caso $i \prec j$, aumenta-se em uma unidade a quantidade de dominadores de j e adiciona-se j à lista de indivíduos dominados por i . Em termos de complexidade temporal, há $O(N^2)$ pares de elementos, e como cada comparação entre indivíduos requer na verdade M comparações, tem-se $O(M \cdot N^2)$. Já para a complexidade em memória, nota-se que é necessário um espaço de ordem $O(N^2)$, por conta das listas de elementos dominados. No pior caso, cada combinação dois a dois implica uma nova adição

na lista. Isso é inferior à abordagem ingênua, em que se usa apenas $O(N)$ em espaço (uma entrada por elemento da população). Assim, é feito um *trade-off* entre espaço e tempo. Vale notar que, considerando um computador padrão, esta complexidade em espaço tende a não ser um problema, posto que para tal acontecer o N necessário implicaria também uma complexidade temporal restritiva, de modo que a abordagem ingênua não seria aplicável na prática.

Em seguida, tem-se a construção das frentes de Pareto, com base nas informações pré-processadas. Enquanto há elementos a serem processados, executa-se a seguinte lógica. Itera-se pelos elementos ainda não alocados em alguma frente. Aqueles que não possuem mais elementos que os dominam (dentre os não processados) são alocados à frente de Pareto atual. Em seguida, para cada elemento alocado na frente atual acessam-se os elementos por ele dominados. Para cada um, decrementa-se em um a quantidade de dominadores - isto é, atualizam-se as quantidades de elementos não processados que dominam os indivíduos restantes. Por fim, aloca-se a fronteira de Pareto atual a uma lista com todas as fronteiras. É importante notar que a divisão dessa fase em duas etapas é deliberada, em função do seguinte problema: caso já fossem atualizadas as contagens de elementos dominantes enquanto é selecionada a frente de Pareto atual, algum indivíduo poderia ser indevidamente alocado em uma frente anterior à qual deveria pertencer. Em termos de memória, o gasto nessa fase é $O(N)$, porque os indivíduos da população não se repetem em mais de uma fronteira F_K . Em termos de tempo, a complexidade é $O(N^2)$. Para o primeiro *loop for* dentro do *while*, o argumento é que há no máximo $O(N)$ iterações (pois é a quantidade máxima de frentes) do *loop* externo, e que o *loop* interno é $O(N)$, passando uma vez por cada indivíduo da população. Para o segundo *loop*, basta notar que a quantidade de elementos na lista S é limitada pela quantidade de pares, que é $O(N^2)$.

A.1 Análise de complexidade do algoritmo ingênuo

No algoritmo ingênuo, fazem-se todas as comparações dois a dois requeridas para todos os níveis de fronteira F_i . Assim, para o pior caso, considere a situação em que cada nível F_i é formado por exatamente um elemento. De maneira mais precisa, para o pior caso, no primeiro nível seriam feitas $\binom{N}{2}$ comparações, no segundo nível seriam $\binom{N-1}{2}$, e assim por diante - cada comparação envolvendo M coordenadas. Logo, a quantidade total de comparações $\#ops$ é dada pela equação A.1.

$$\#ops = M \cdot \sum_{k=1}^N \binom{k}{2} = M \cdot \sum_{k=1}^N \frac{k \cdot (k-1)}{2} = \frac{M}{2} \cdot \left(\sum_{k=1}^N k^2 - \sum_{k=1}^N k \right) \quad (A.1)$$

Ainda, tem-se as identidades apresentadas em A.2 que mostram que $O(S_2) = O(N^3)$

e $O(S_1) = O(N^2)$.

$$\begin{aligned} S_2 &= \left(\sum_{k=1}^N k^2 \right) = \left(\frac{N(N+1)(2N+1)}{6} \right) \\ S_1 &= \left(\sum_{k=1}^N k \right) = \left(\frac{N(N+1)}{2} \right) \end{aligned} \tag{A.2}$$

Logo, juntando [A.2](#) e [A.1](#) em [A.3](#) que o método ingênuo é $O(M \cdot N^3)$.

$$O(\#ops) = O(M \cdot (S_2 - S_1)) = O(M \cdot N^3 - M \cdot N^2) = O(M \cdot N^2) \tag{A.3}$$